

AUTOMATIZACIÓN E IA CON LOVABLE Y N8N

TEMA 4. Automatización Avanzada en n8n

icono TRAINING CONSULTING

Contenidos Tema 4

/ 1. Más integraciones con Apps Externas

/ 2. Uso de nodos If, Switch, Merge y Wait

/ 3. Bucles y ramificaciones: iteración eficiente

/ 4. Manejo de errores: configuración de "error workflows"

/ 5. Webhooks bidireccionales

/ 6. IA aplicada a n8n: Modelos de Machine Learning

/ 7. Agentes IA orquestadores en n8n

DISCORD: OPCIONES DE CONEXIÓN DESDE N8N

Discord es una plataforma de creación de comunidades y comunicación multifuncional mediante texto, voz y vídeo.



OAuth2 (Simplificado)

- **Aconsejado:** Envíos puntuales, notificaciones y alertas simples.
- **Desaconsejado:** Leer mensajes, escuchar canales o moderar usuarios.
- **Limitación:** Las acciones suelen quedar vinculadas a tu usuario personal.

Bot API (Avanzado)

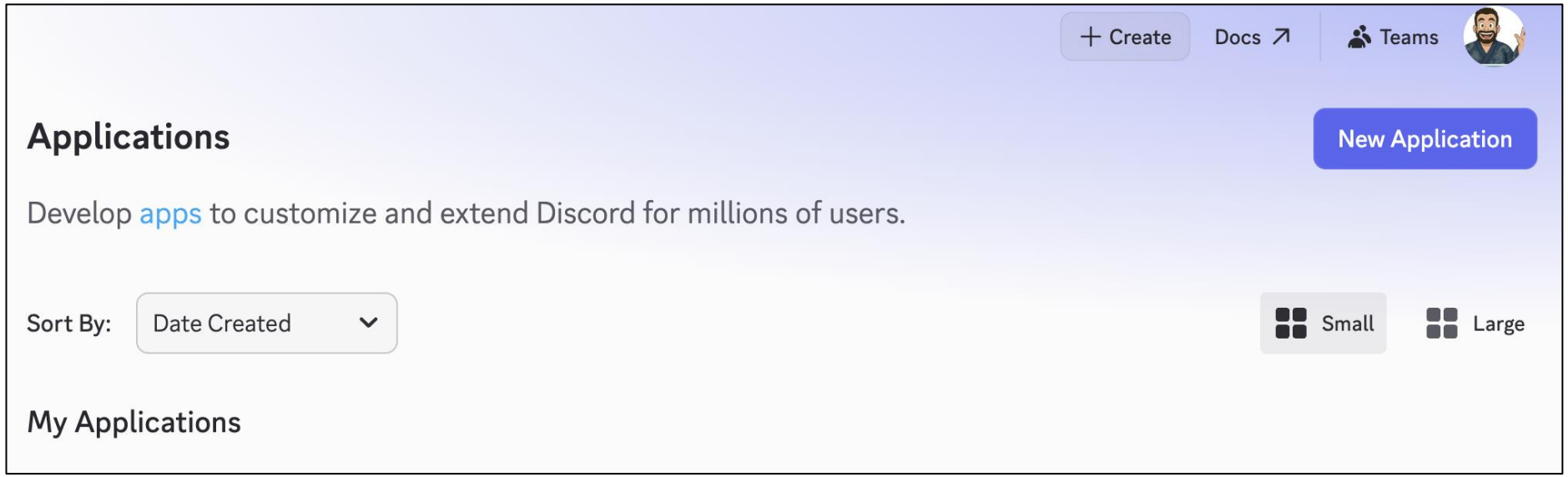
- **Aconsejado:** Escucha continua 24/7, moderación y gestión de roles.
- **Desaconsejado:** Tareas rápidas sin querer lidiar con permisos técnicos.
- **Ventaja:** El bot tiene identidad independiente y permisos aislados.

En el ejercicio práctico, probaremos principalmente el método simplificado que es nuevo desde Octubre 2026. No obstante, aquí te explicaré también el proceso para el método avanzado, mediante Bot API

CONEXIÓN A DISCORD: CREACIÓN DE LA CREDENCIAL (*Bot API*, método avanzado)

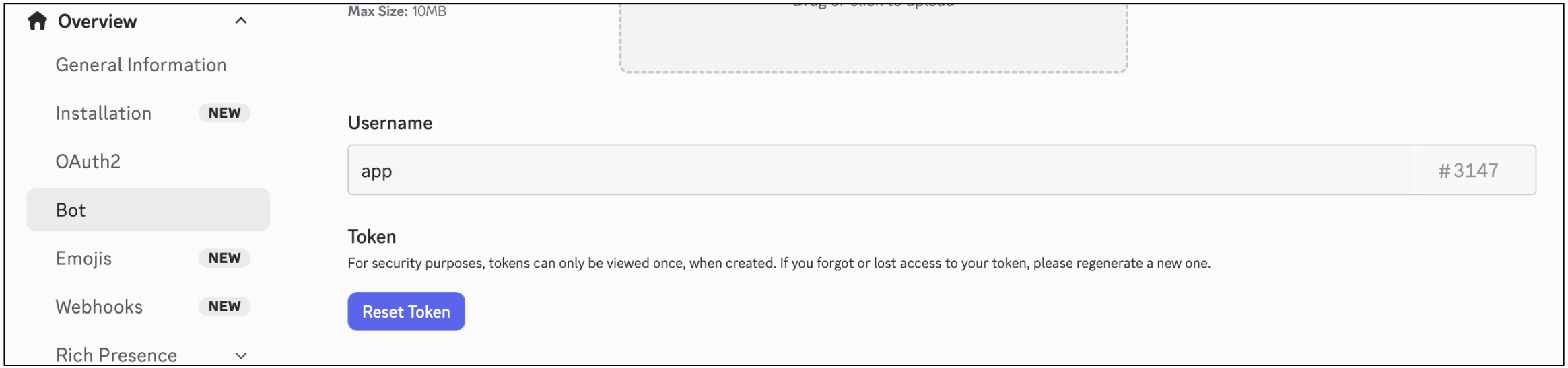
En la Web de Discord, ir a: https://discord.com/developers/applications

- Crear una nueva aplicación



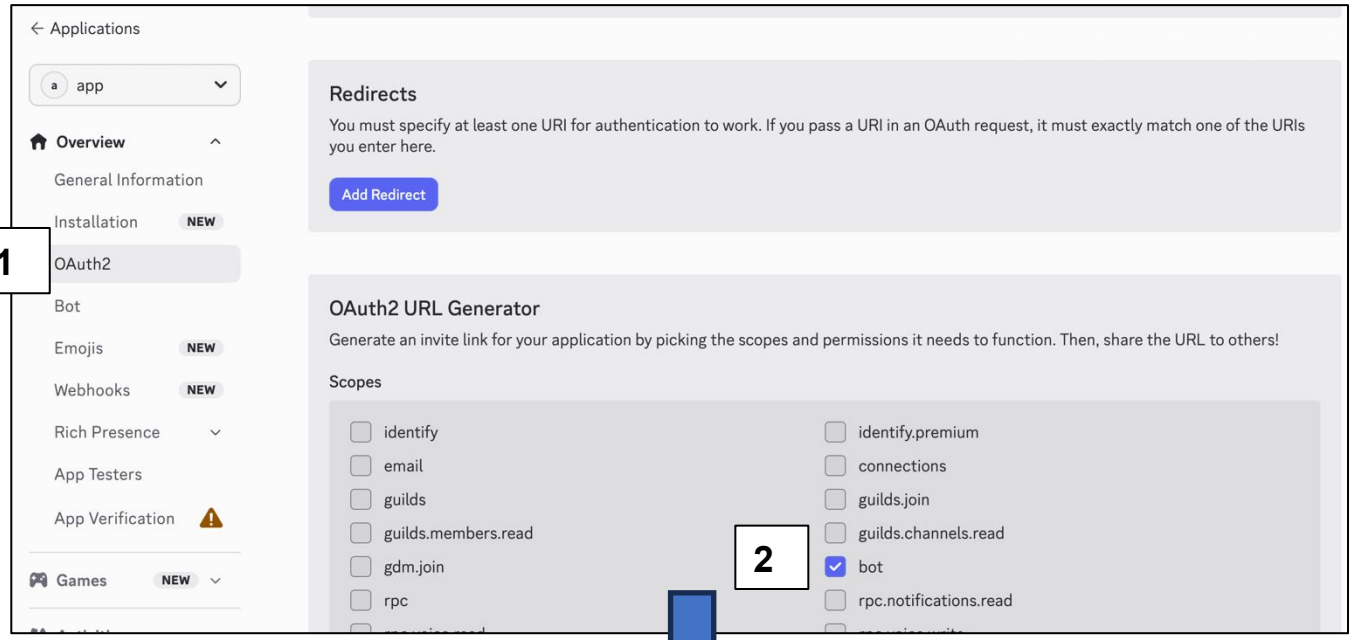
CONEXIÓN A DISCORD: CREACIÓN DE LA CREDENCIAL (cont.)

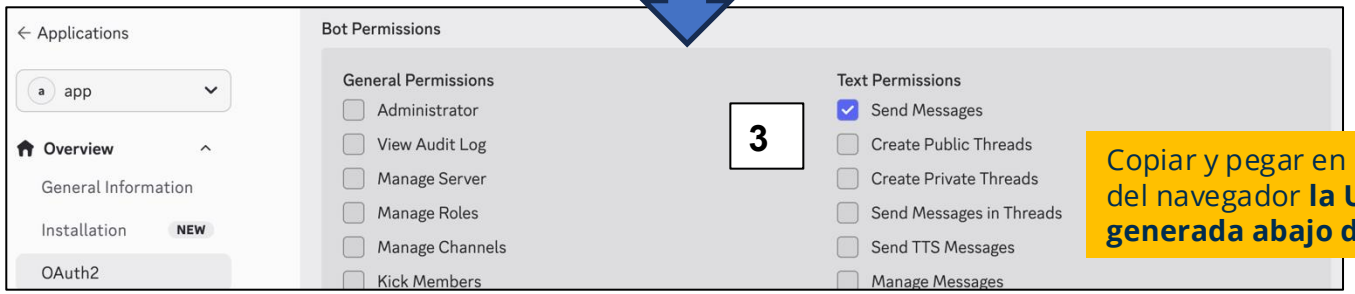
- En la aplicación que se está creado, ir a "**Bot**" a la izquierda, y "**Reset Token**"
- Copiar el nuevo token y llevarlo a **n8n**, para crear una credencial del tipo "**Bot Token**".



CONFIGURACIÓN DEL BOT (cont.)

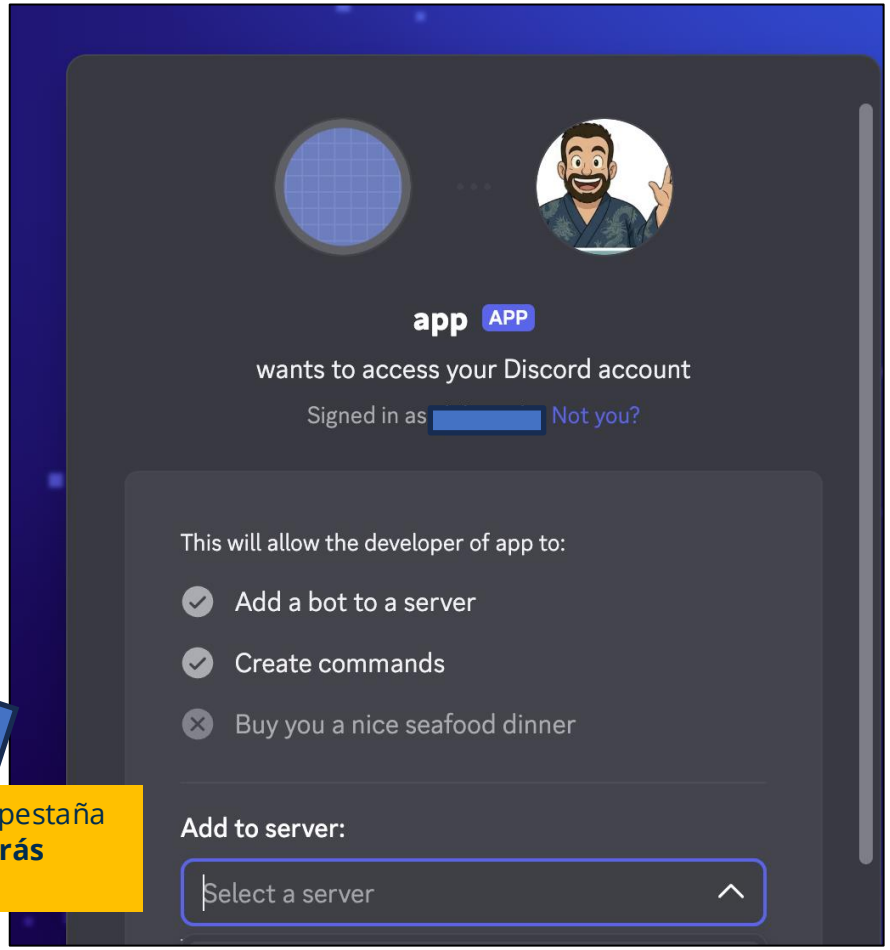
- Asignación de permisos a nuestro bot.





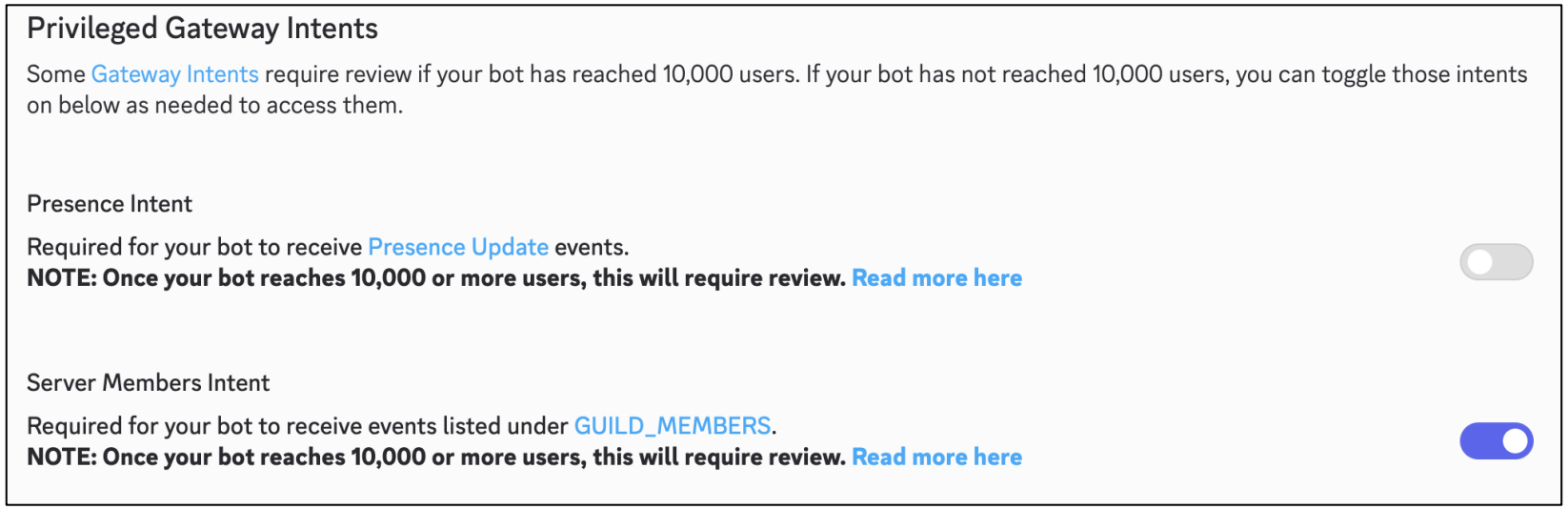
Copiar y pegar en una nueva pestaña del navegador **la URL que verás generada abajo del todo**

4

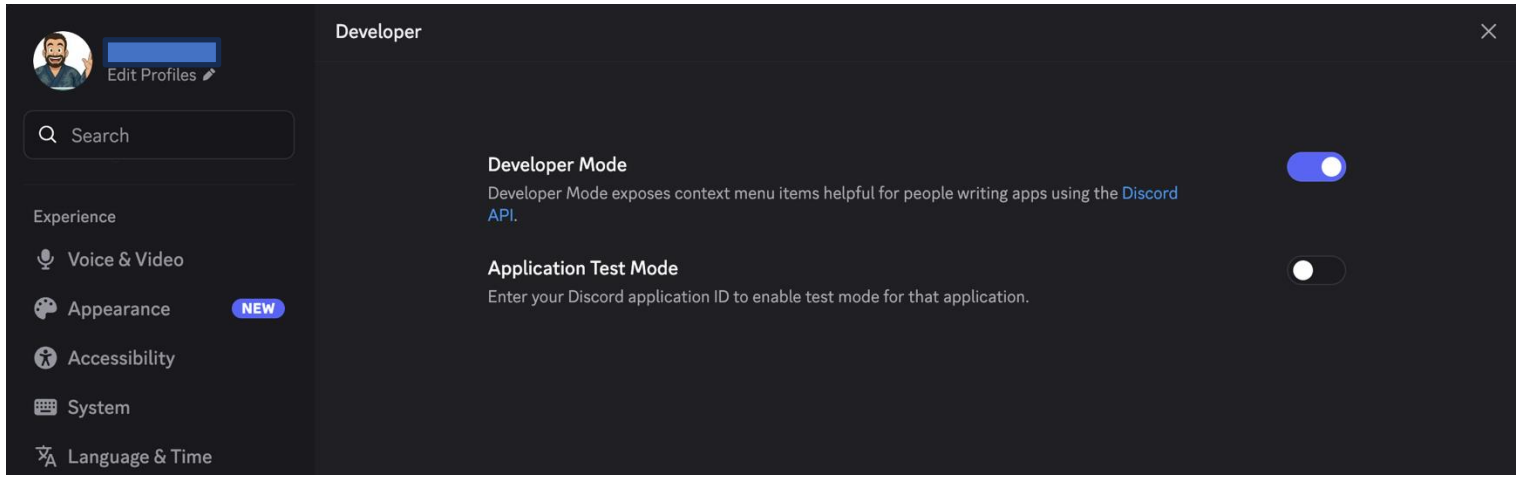


SI DESDE N8N QUIERES ENVIAR MENSAJES A UN USUARIO (Y NO A UN CANAL...)

- **Activar "Server Members Intent" en el Bot, pestaña "Bot" de tu aplicación**



- Por último, en ajustes de tu perfil de Discord, sección "Developer", activar "**Developer Mode**". Esto permitrá copiar tu user ID y también seleccionar desde el nodo en n8n al usuario al que enviarle un mensaje actuando como el bot.



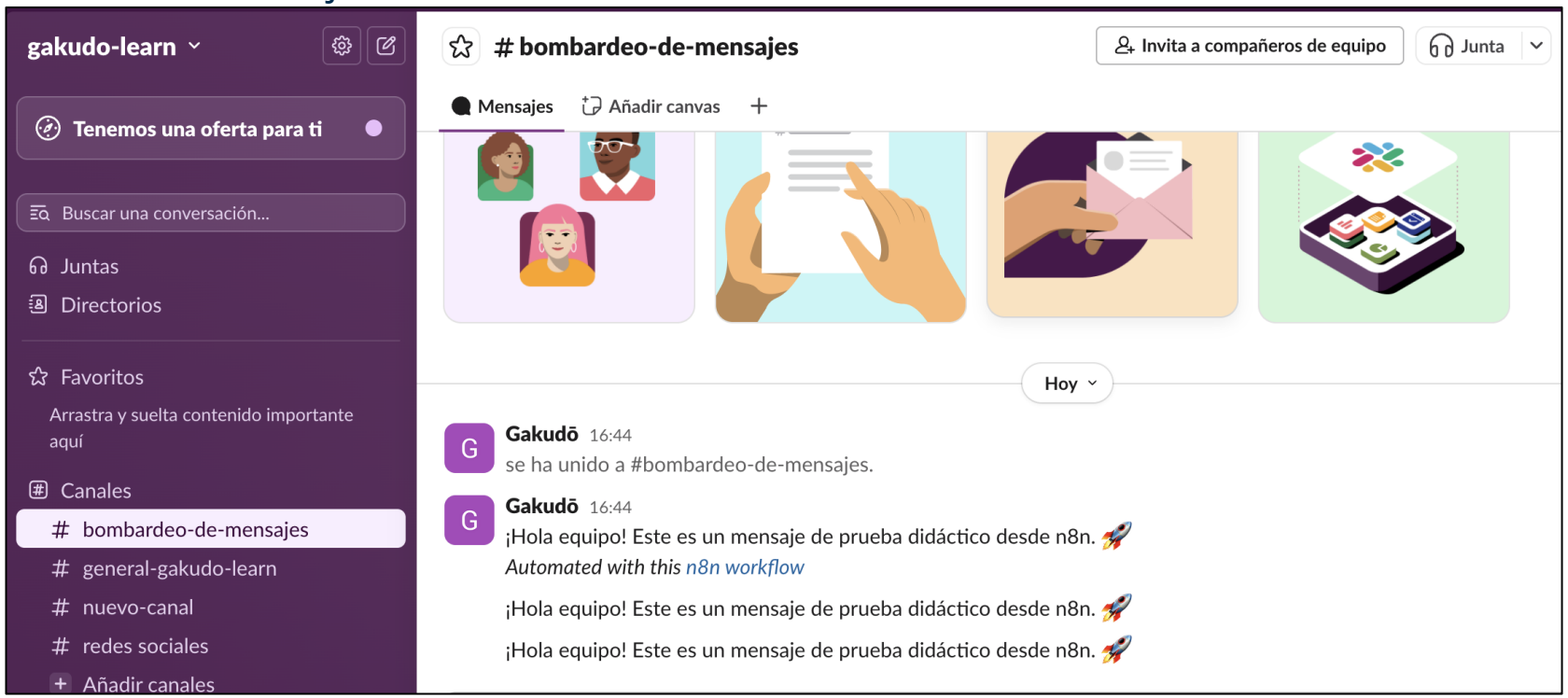
CONEXIÓN A SLACK: CREACIÓN DE LA CREDENCIAL

En Slack, el proceso es infinitamente más fácil. Para definir la credencial, elegirás unirte a un espacio.

- Antes de ello, te invito a unirte a este espacio de Slack que he creado para que todos lo probemos.

https://gakudo-learn.slack.com

- Para tu nodo de Slack en n8n, elige "Resource: Message", "Operation: Send", y elegir el canal #bombardeo-de mensajes:



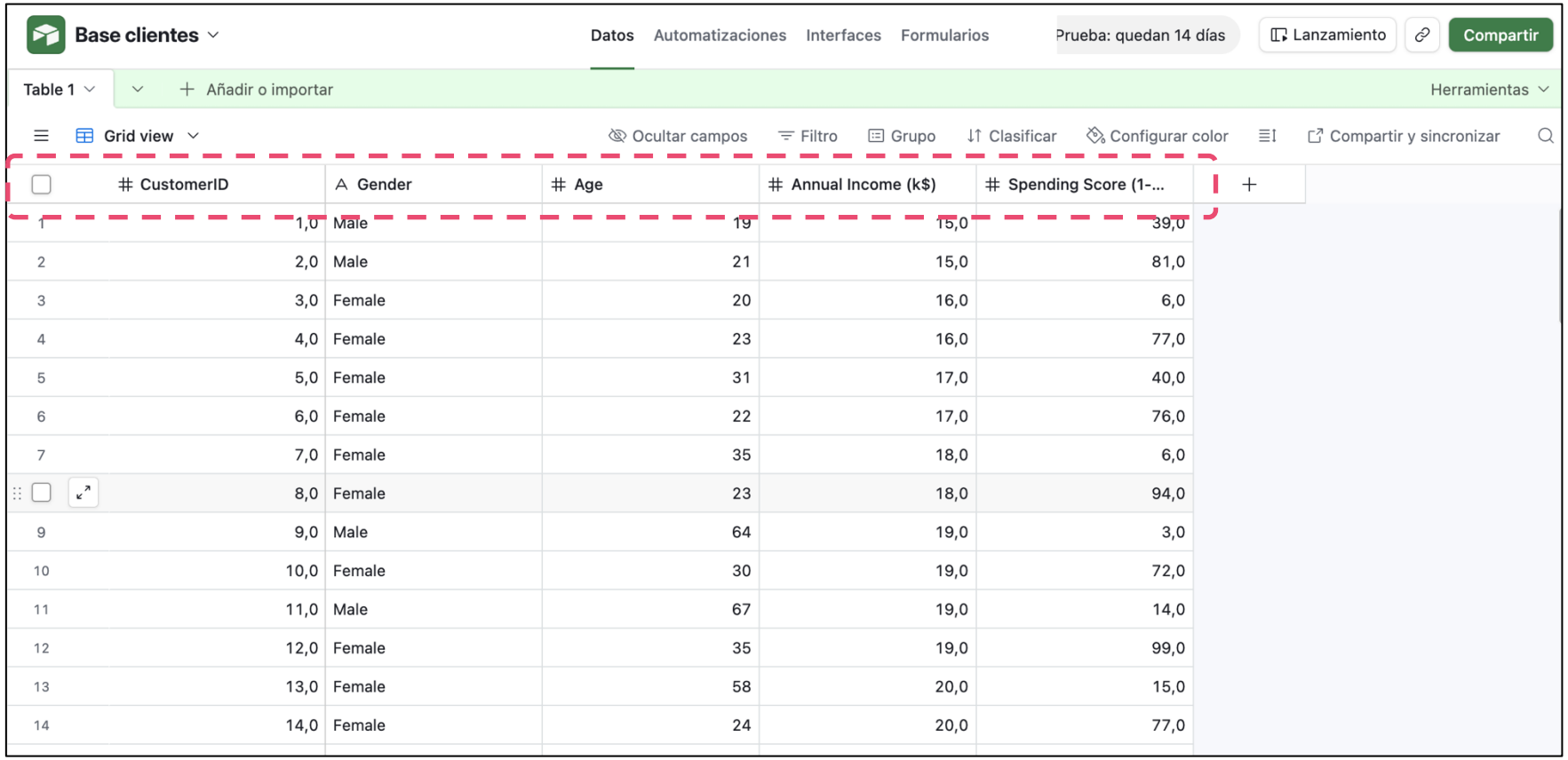
DEFINIENDO UNA CREDENCIAL PARA AIRTABLE MEDIANTE TOKEN DE ACCESO PERSONAL

- Permisos **(ámbitos de aplicación)** necesarios.
- **Acceso:** recursos (espacios de trabajo, BBDDs) a los que puede acceder este token.



EJERCICIO PRÁCTICO

- Poblar una tabla previamente creada en Airtable con datos provenientes de n8n
- **IMPORTANTE:** asegúrate en primer lugar de crear la tabla vacía, y actualizar las columnas y sus tipos para que concuerden con los datos de clientes.

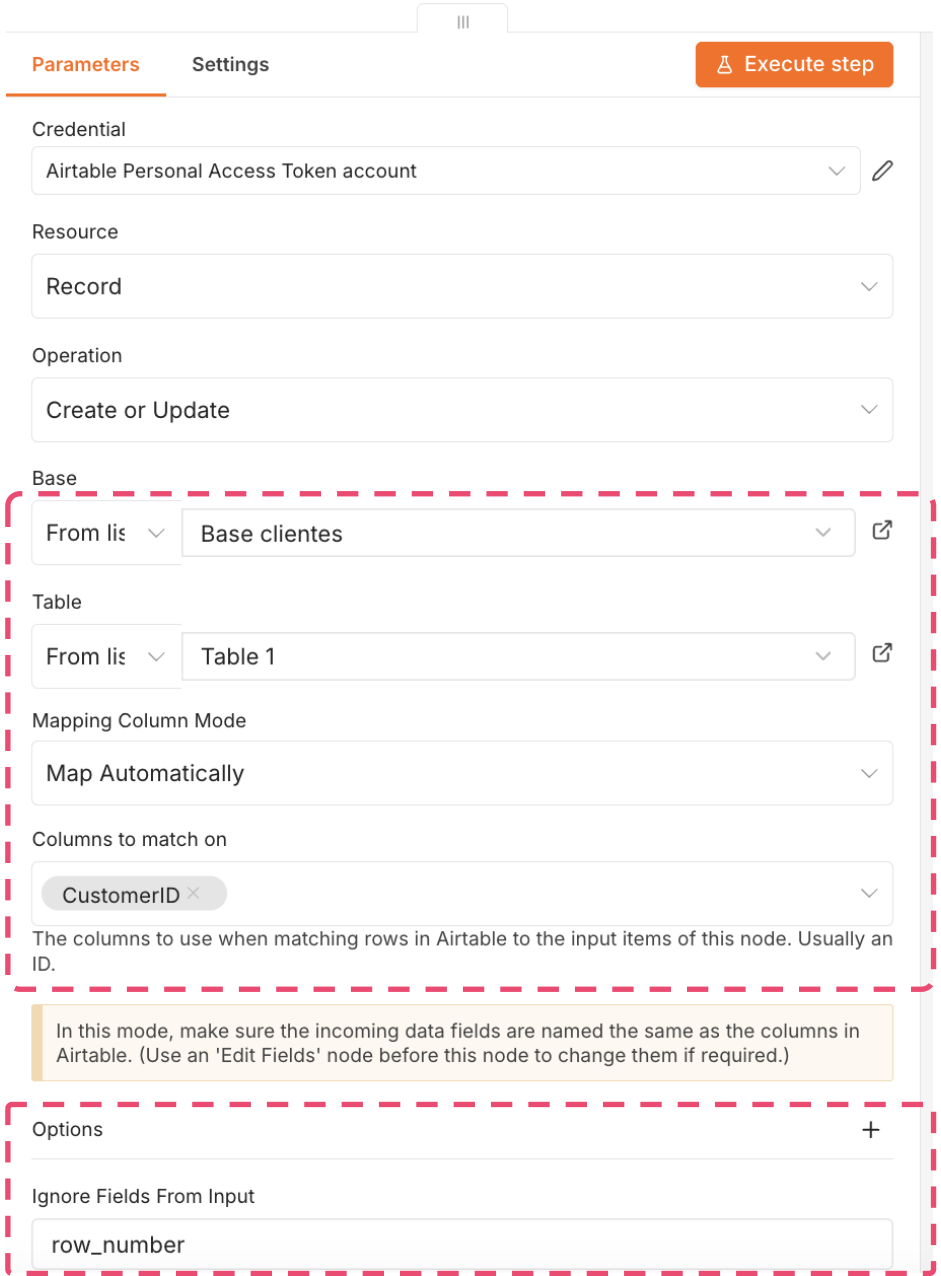


	CustomerID	Gender	Age	Annual Income (k$)	Spending Score (1-...
1	1,0	Male	19	15,0	39,0
2	2,0	Male	21	15,0	81,0
3	3,0	Female	20	16,0	6,0
4	4,0	Female	23	16,0	77,0
5	5,0	Female	31	17,0	40,0
6	6,0	Female	22	17,0	76,0
7	7,0	Female	35	18,0	6,0
8	8,0	Female	23	18,0	94,0
9	9,0	Male	64	19,0	3,0
10	10,0	Female	30	19,0	72,0
11	11,0	Male	67	19,0	14,0
12	12,0	Female	35	19,0	99,0
13	13,0	Female	58	20,0	15,0
14	14,0	Female	24	20,0	77,0

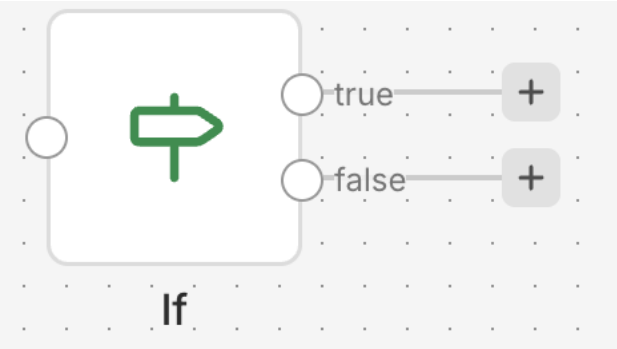
Airtable

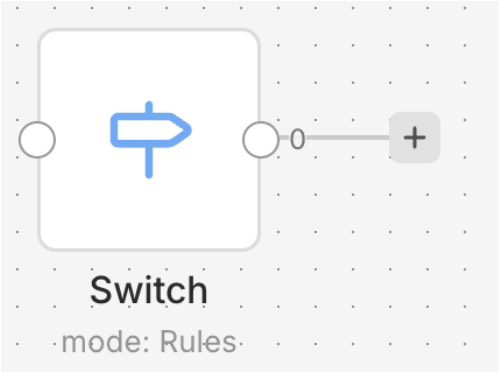
EJERCICIO PRÁCTICO

- Poblar una tabla previamente creada en Airtable con datos provenientes de n8n
- **Configuración del nodo en n8n**



Toma de Decisiones: "If" vs. "Switch"





1. Nodo IF (La bifurcación simple)

•**Lógica:** Verdadero o Falso (Sí/No).

•**Usar para:** Validaciones rápidas de si algo se cumple o no.

- *Ejemplo:* "¿El cliente gastó más de 100€?
- **SÍ:** Enviar cupón
- **NO:** Enviar "gracias".

En n8n, el IF permite encadenar múltiples condiciones (AND/OR), pero siempre termina el resultado final siempre será Verdadero o Falso.

2. Nodo SWITCH: múltiples caminos

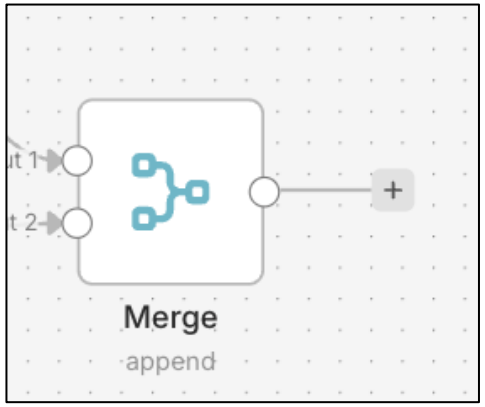
- **Lógica:** Basada en reglas específicas (1 entrada, múltiple posibles salidas).

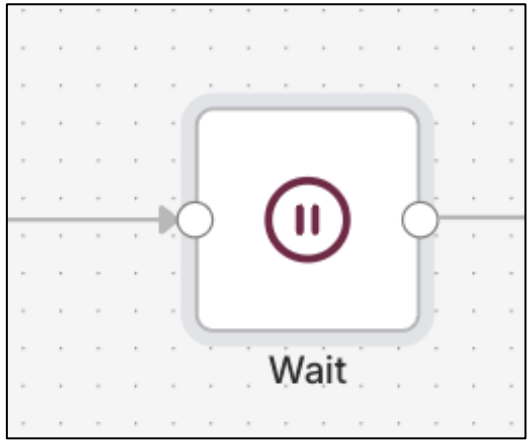
•**Uso ideal:** Categorización por etiquetas o estados.

- *Ejemplo:* Según el país del formulario: "ES" → Ruta 1, "MX" → Ruta 2, "US" → Ruta 3.

• **Ahorro "Free":** Evita encadenar 5 nodos IF seguidos, lo que hace el flujo más legible y fácil de debuguear.

Unión de flujos (Merge) y Control del Tiempo (Wait)





•1. Nodo Merge: unir datos de distintas fuentes

• **Lógica:** Combina datos de dos ramas diferentes en una sola.

•Modos clave:

- **Append:** Junta una lista debajo de la otra. Es como enganchar dos trenes, uno tras otro.
- **Combine:** Une información de dos fuentes sobre el mismo objeto, similar a los JOIN de bases de datos relacionales.

•Uso ideal: Cuando necesitamos que dos caminos distintos terminen en la misma acción final (ej. enviar un solo correo con datos de diferentes tipos de viviendas a promocionar).

2. Nodo WAIT: gestión de esperas

•Lógica: Detiene la ejecución por un tiempo (segundos, horas, días) o hasta una fecha específica.

•Uso ideal: Seguimientos (Follow-ups).

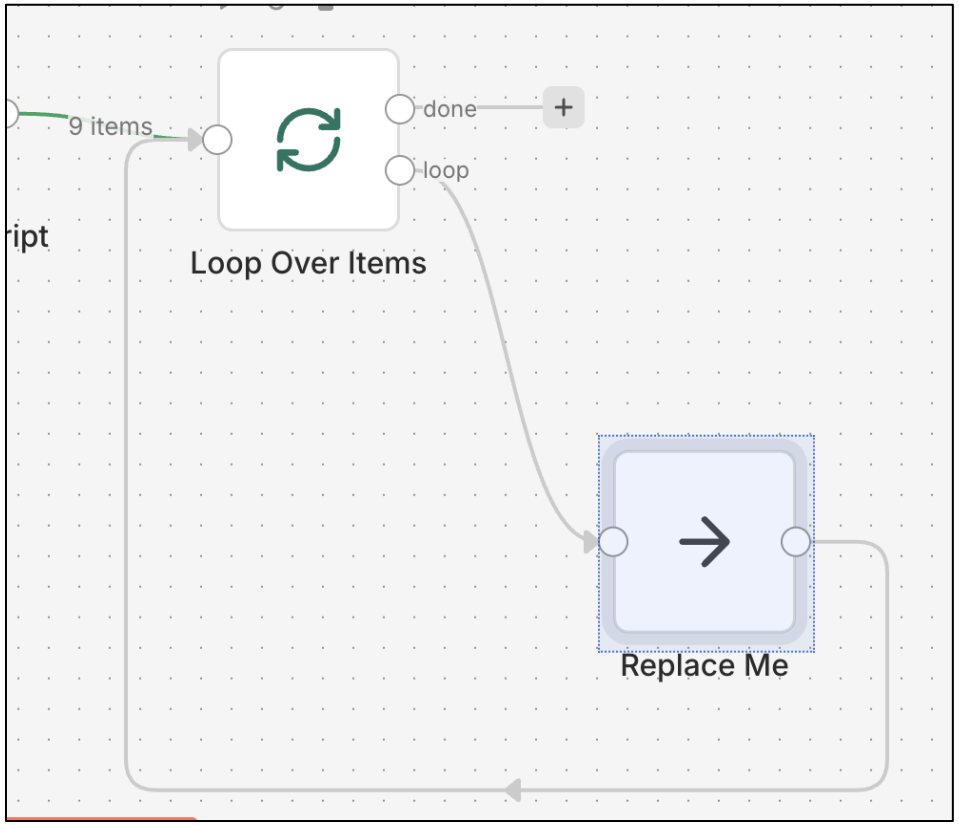
- *Ejemplo:* Envías un presupuesto → **Wait 24h** → Revisar si se ha respondido.

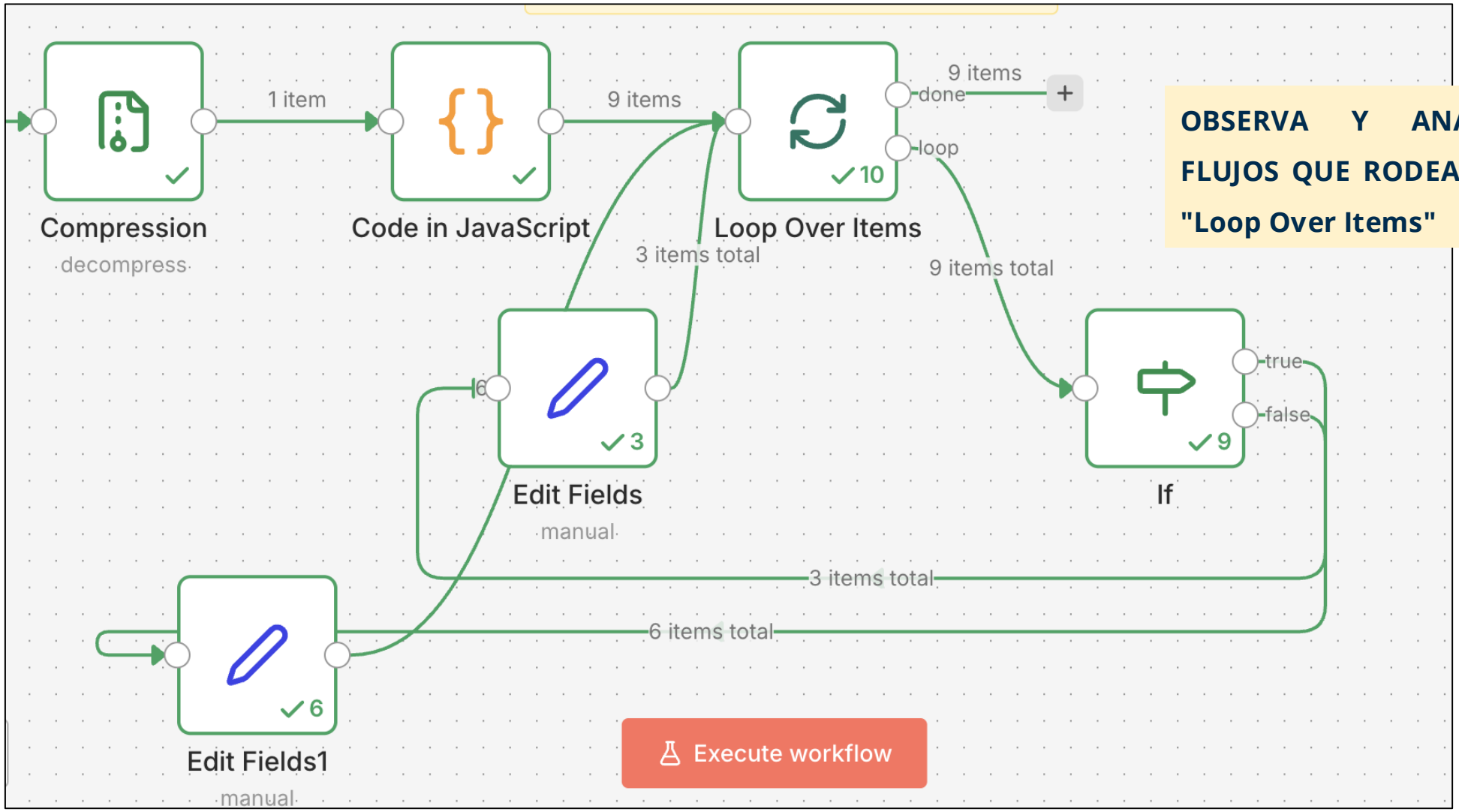
•Nota importante: En versiones self-hosted de n8n, el flujo "duerme" en la base de datos, no se consumen recursos de cómputo mientras espera.

El nodo "Loop Over Items"

- Itera múltiples veces sobre cada uno de los items de la colección de datos (objetos JSON) de entrada.
- Dentro del bucle, se añade un "nodo comodín" llamado *Replace Me*, que deberemos eliminar y reemplazar por aquellas operaciones que queramos realizar iteractivamente sobre cada item.

IMPORTANTE: si dentro del bucle creamos ramificaciones, p.ej. Las resultantes de un nodo If o Switch, hay que reconectar todas las posibles bifurcaciones de nuevo con el nodo "Loop Over Item" en su lado izquierdo, para que el bucle quede correctamente cerrado.





OBSERVA Y ANALIZA LOS FLUJOS QUE RODEAN AL NODO "Loop Over Items"

Resiliencia y control en n8n

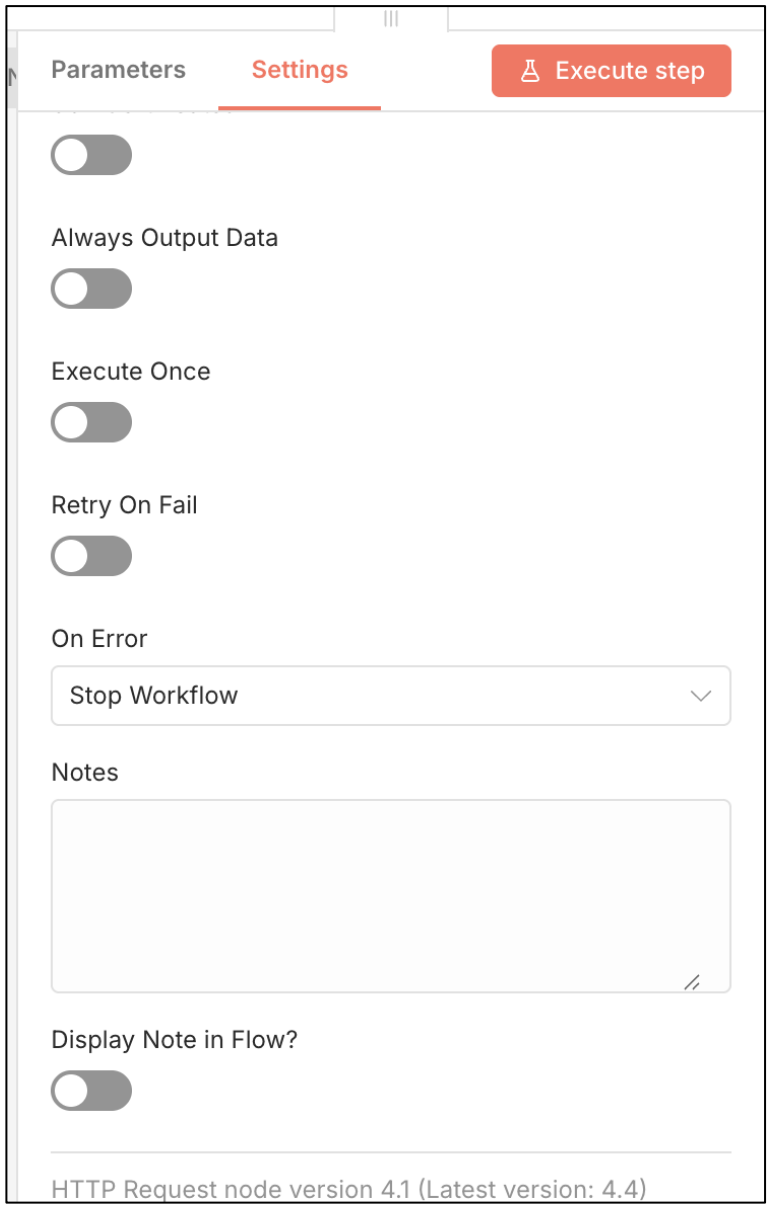
El manejo de errores puede añadir robustez a un flujo inicialmente frágil, especialmente en contextos empresariales.

Para ello, n8n ofrece dos "mecanismos de defensa":

[1] Configuración a nivel de nodo: decidir dentro de un nodo qué hacer cuando algo sale mal.

- Opción "*On Error*": se puede detener el flujo, como sucede por defecto, o bien continuar como si nada hubiera pasado.
- También hay una opción "*retry on failure*" para reintentar la ejecución – útil cuando el error puede provenir por factores externos.

[2] A nivel de flujo: se trata de crear un flujo especial, e independiente, con un nodo especial llamado **Error Trigger.** A nivel avanzado, en flujos en producción es bueno vincularlos a un "Error workflow centralizado".



Un Webhook dinámico aporta dos grandes valores añadidos:

- No solo se recibe el dato –al igual que en los Webhook convencionales – sino que devuelve al sistema de origen una respuesta personalizada, por ejemplo, si un nombre de usuario enviado por formulario existe o no en una BBDD.

El Webhook tradicional

- Flujo de datos unidireccional. El sistema exterior envía datos a n8n para iniciar el flujo.
- Ejemplo: enviar un comando *curl* a través de la terminal.

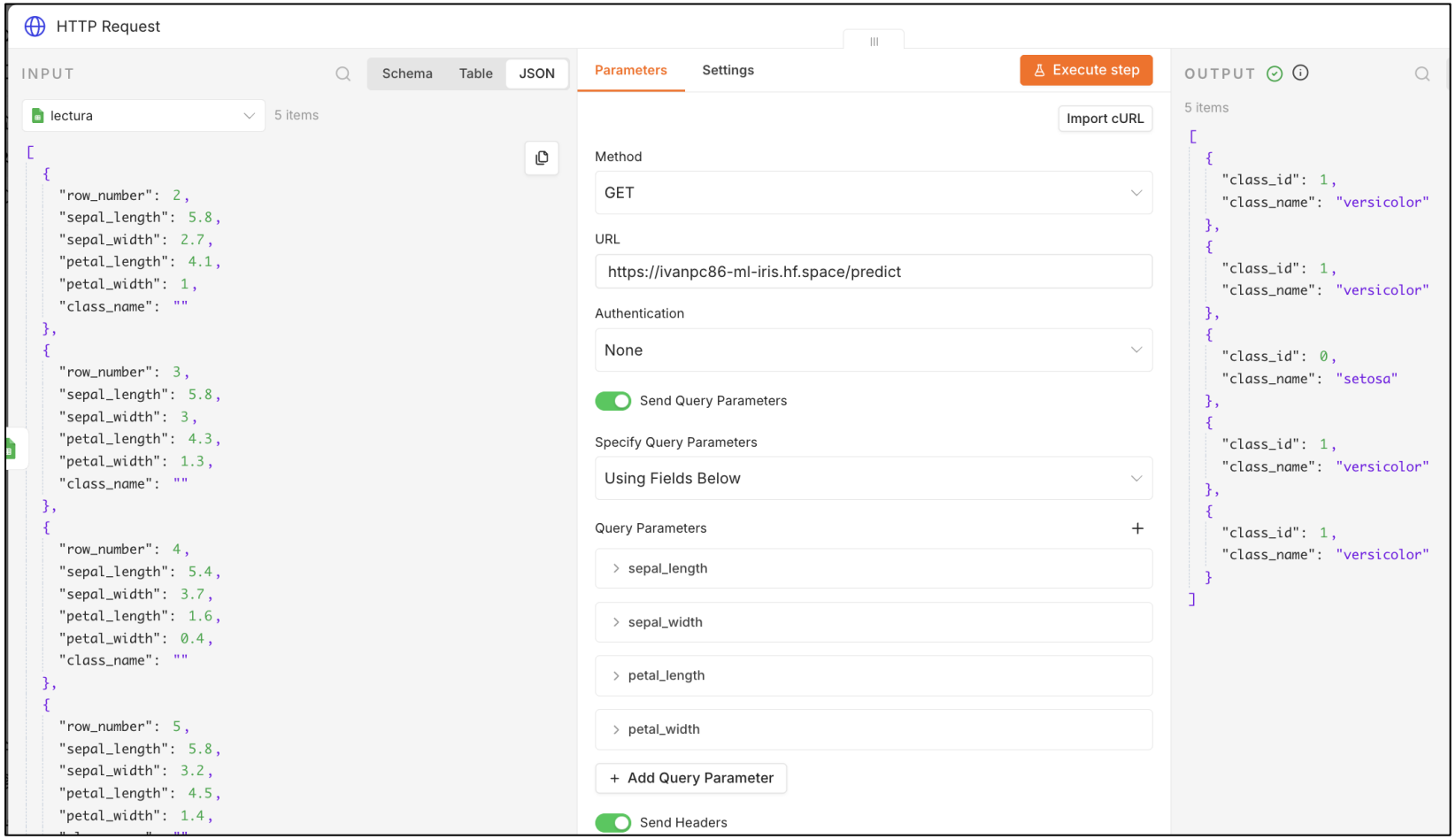
El Webhook dinámico

- Múltiples configuraciones dependientes del contexto.
- Flujo de datos bidireccional: n8n responde al instante al sistema externo, p.ej. JSON dinámico con los resultados de una validación.



La clave para "hablar" (desde n8n) con modelos de Machine Learning para clasificación o regresión desplegados en la Web:

- Nodo HTTP Request con método GET



He desplegado un modelo de clasificación de flores iris para ti en: https://ivanpc86-ml-iris.hf.space/predict

Puedes probarlo primero y ver los parámetros que necesita como datos de entrada en: https://ivanpc86-ml-iris.hf.space/docs

Asegúrate de "mapear" tus datos entrantes al nodo con los parámetros del modelo, mediante "**Query Parameters**"

Hugging Face Spaces

- Hugging Face Spaces permite alojar modelos de Machine Learning entrenados por ti en Python de forma gratuita.
- **FastAPI** actúa como "puente" entre el modelo alojado en HF Spaces, y el exterior. Crea la ruta (/predict) y se queda permanentemente escuchando peticiones entrantes.
- **La Respuesta:** FastAPI procesa el dato, se lo pasa al modelo y éste devuelve (a n8n, por ejemplo) la(s) predicción(es) en un formato amigable y estructurado (JSON).



Modelos de Lenguaje

Un **modelo de lenguaje** es un sistema capaz de comprender y generar texto de manera coherente a partir de patrones aprendidos.

Es **un tipo específico de modelo de NLP** cuyo propósito central es **predecir y generar secuencias de texto**, además de **comprenderlas** en cierto grado. Tienen una capacidad de **aprendizaje a nivel probabilístico** sobre el lenguaje.

Sistema de NLP:
Conjunto más amplio de técnicas, modelos o herramientas diseñadas para realizar tareas sobre lenguaje natural (clasificación, resumen, extracción de entidades, análisis de sentimiento, traducción, etc.). Un sistema de NLP **puede usar o no un modelo de lenguaje** como parte de su arquitectura.

Modelos de Lenguaje

"Un **modelo de lenguaje** es un sistema capaz de comprender y generar texto de manera coherente a partir de patrones aprendidos."

Es **un tipo específico de modelo de NLP** cuyo propósito central es **predecir y generar secuencias de texto**, además de **comprenderlas** en cierto grado. Tienen una capacidad de **aprendizaje a nivel probabilístico** sobre el lenguaje.

Generar lenguaje

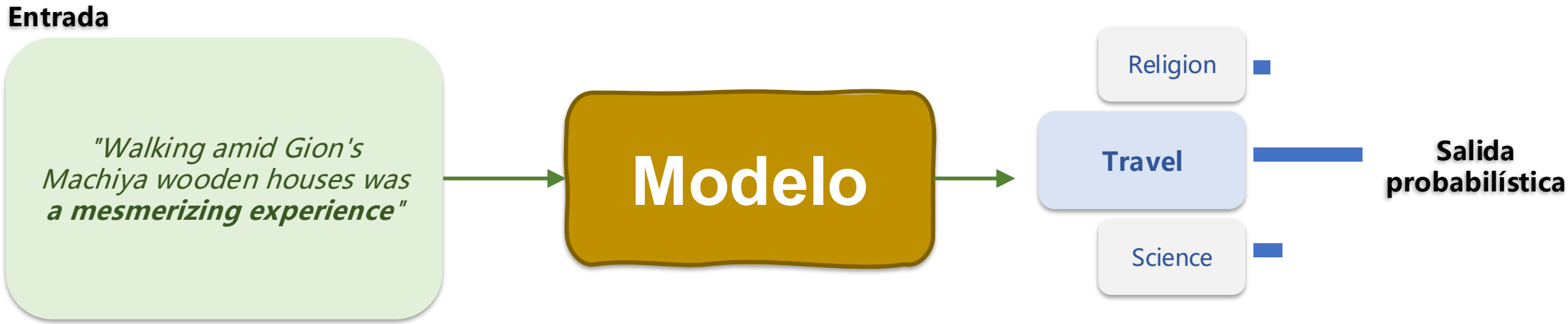
- Generación de texto
- Generación de código

Entender lenguaje

- Clasificación de texto y análisis de sentimientos
- Traducción automática
- Resumen de texto
- Reconocimiento de intenciones
- Respuestas a preguntas
- Reconocimiento de entidades nombradas (Named Entity Recognition)

Clasificación de texto

La tarea en la que un modelo de lenguaje (u sistema de NLP de otro tipo) asigna una (o a veces varias) categoría predefinida a un texto

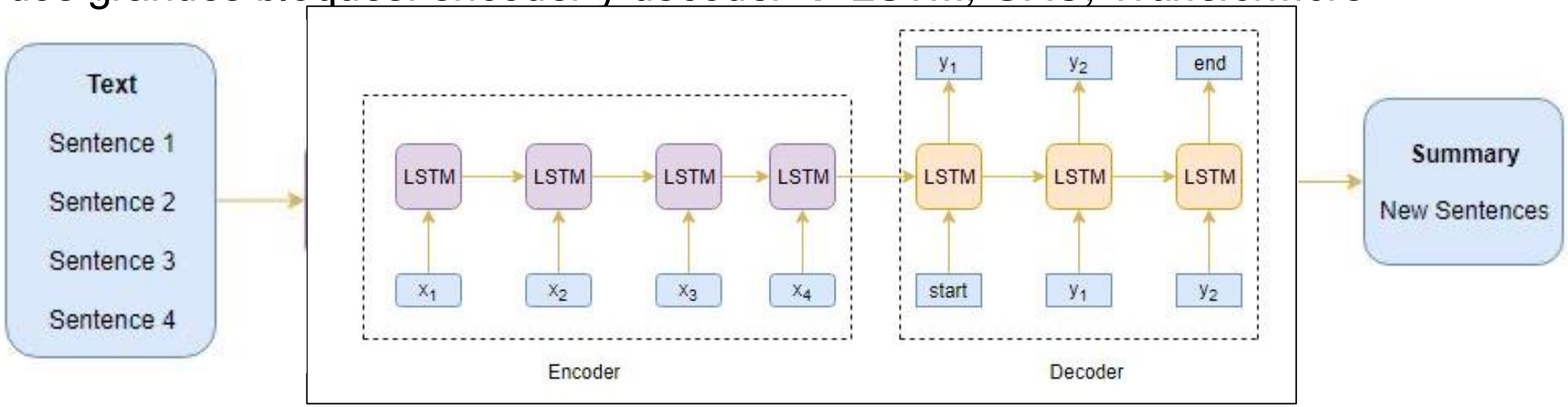


Resumen de Texto

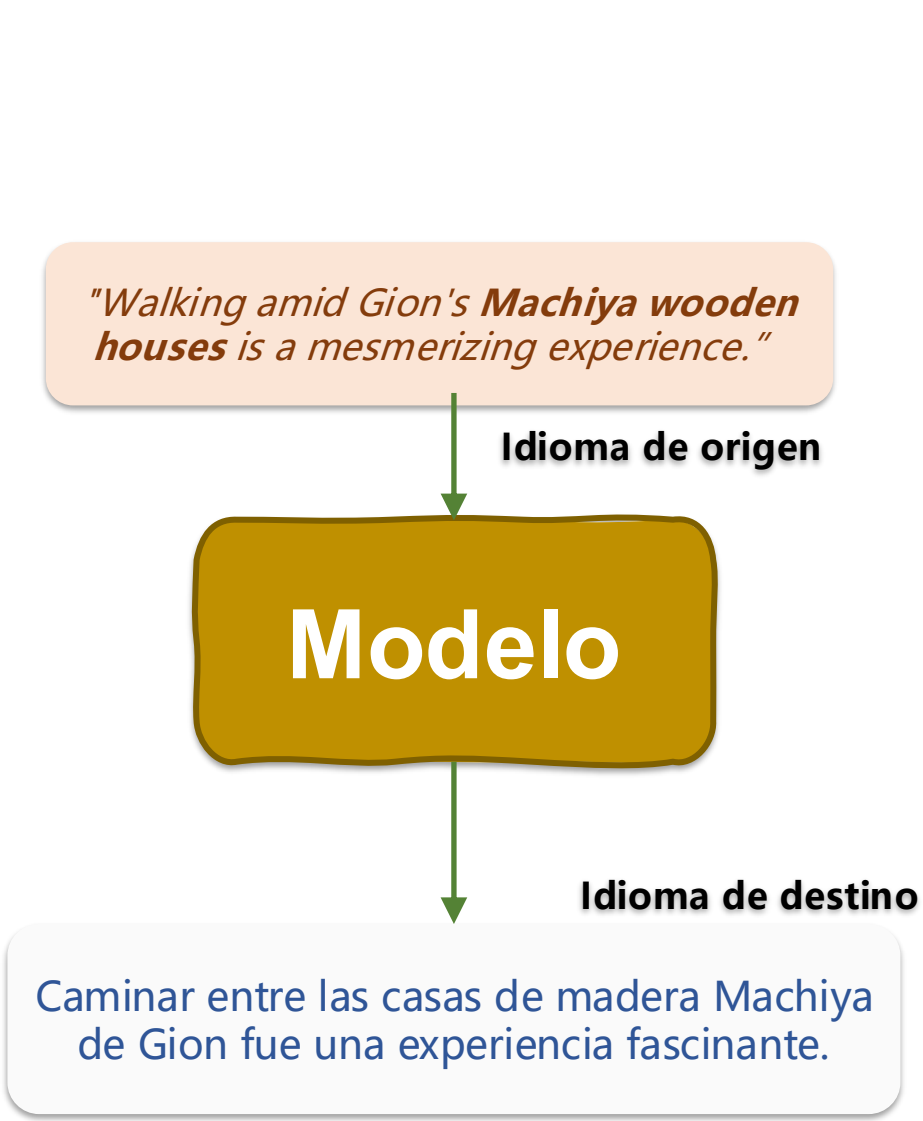
Inferir el contexto e información clave de un texto para crear una versión resumida del mismo, preservando la información y semántica principales

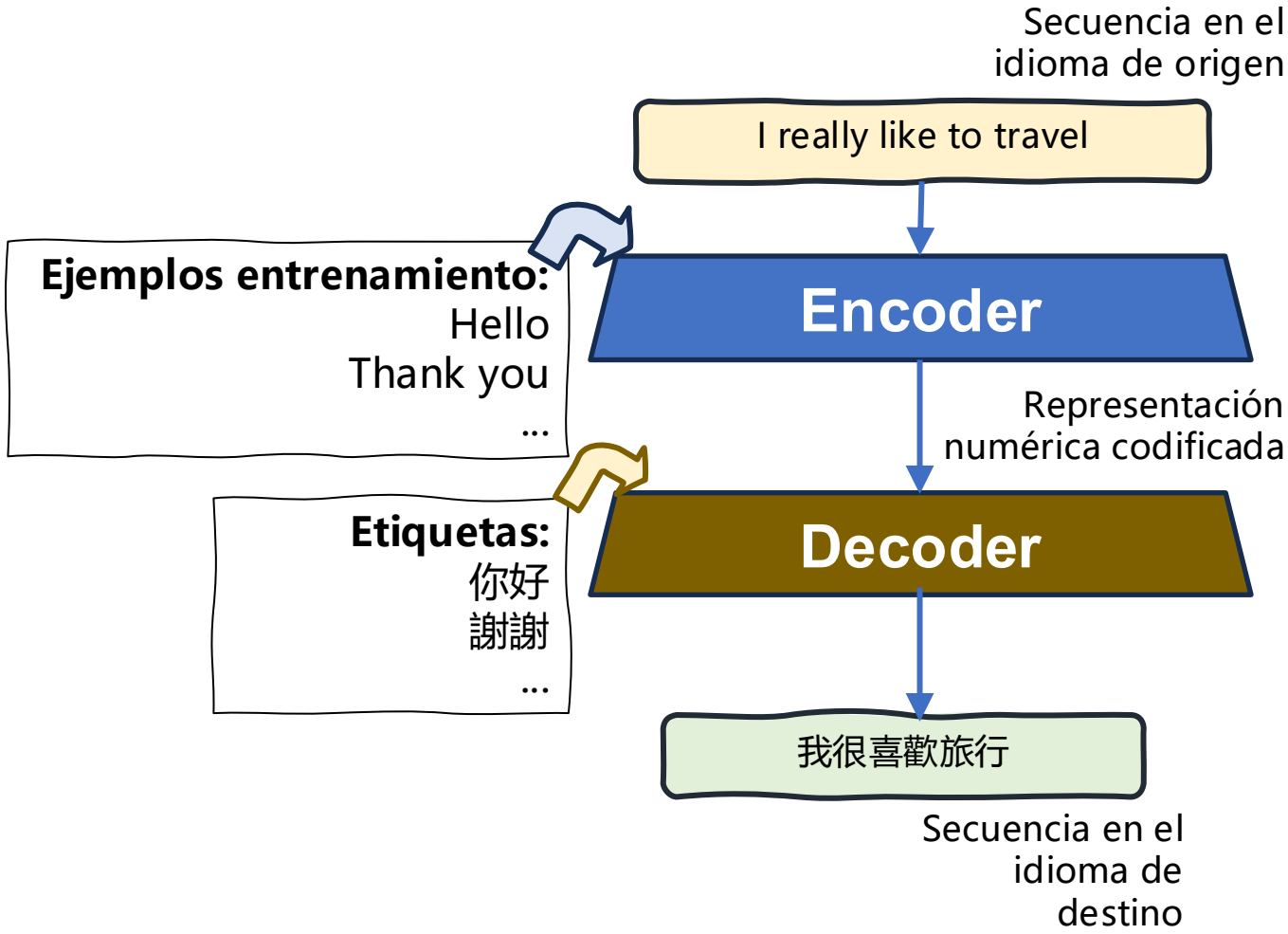
Sequence to sequence (seq2seq) problem: la entrada y la salida son ambas secuencias de texto, de diferente longitud.

- Se resuelven con arquitecturas especializadas de deep learning que consisten en dos grandes bloques: encoder y decoder → *LSTM, GRU, Transformers*



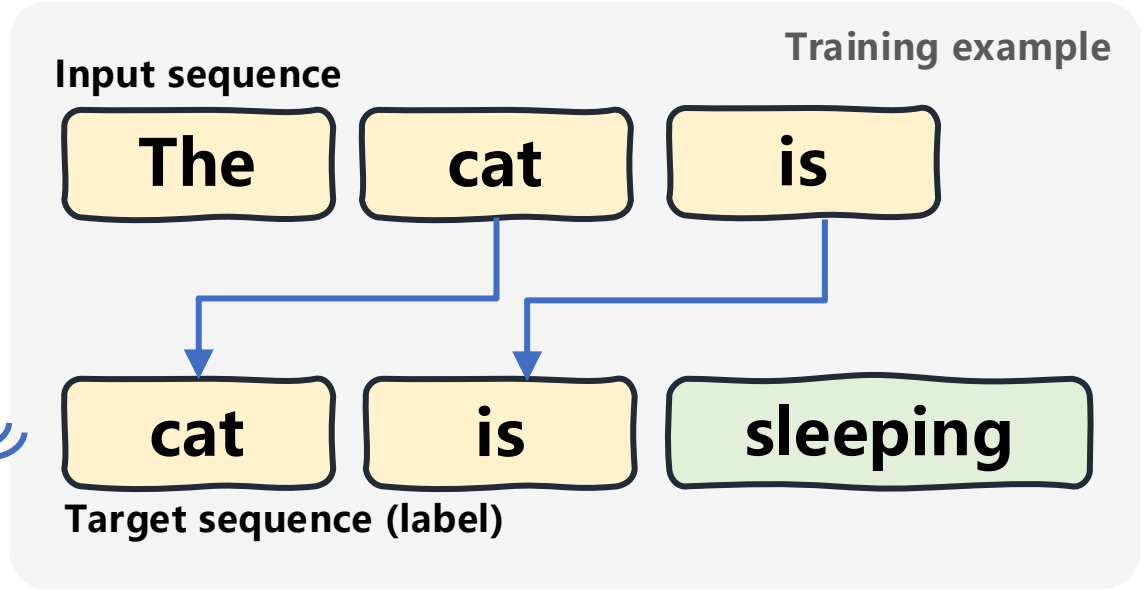
Traducción automática

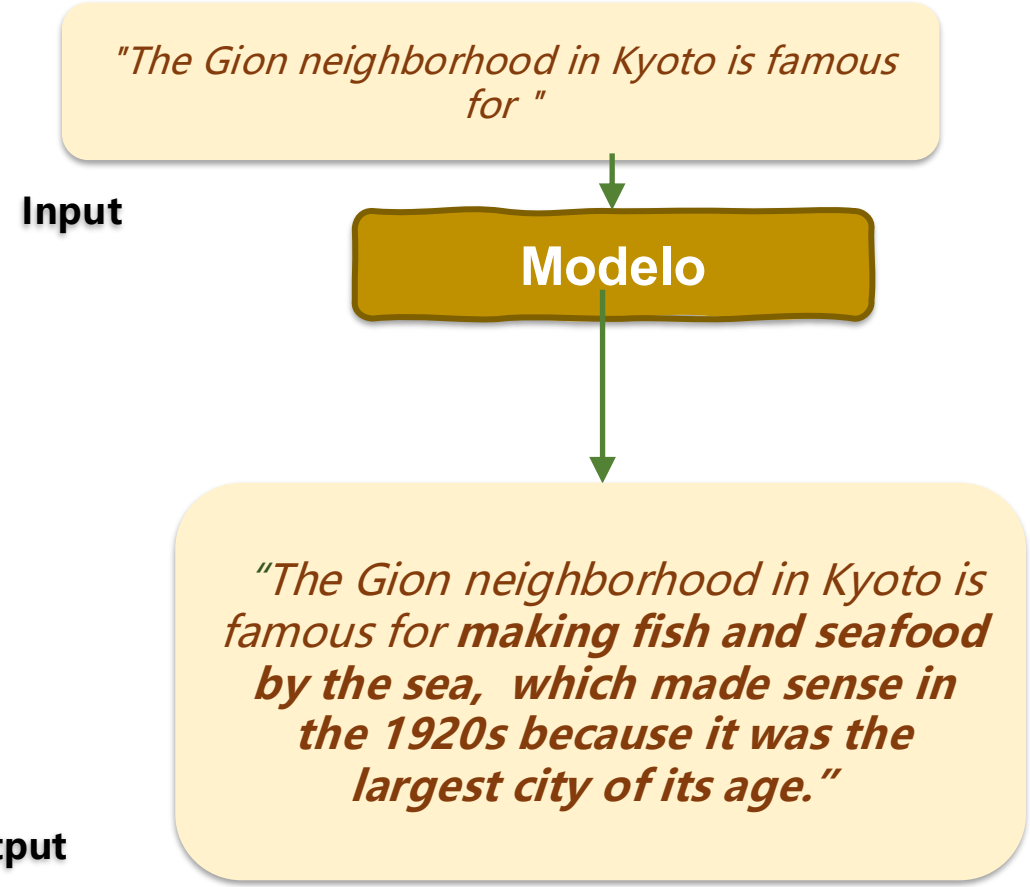




Generación de texto

Generación de secuencia a partir de una entrada (*prompt)*, generando iterativamente la palabra más “probable” de ser la siguiente en la respuesta (entre miles de palabras)





The

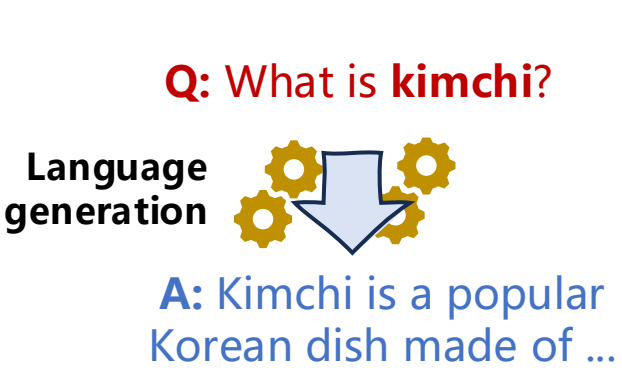
Respuestas a Preguntas (Question-Answering)

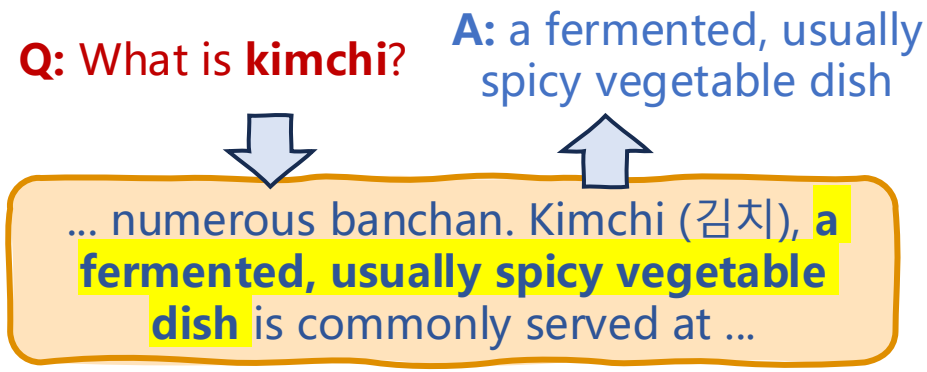
QA es una tarea que puede ser resuelta de forma extractiva o generativa

Dos entradas:

- **Contexto**: el texto sobre el que se formula la pregunta

- La propia **pregunta**



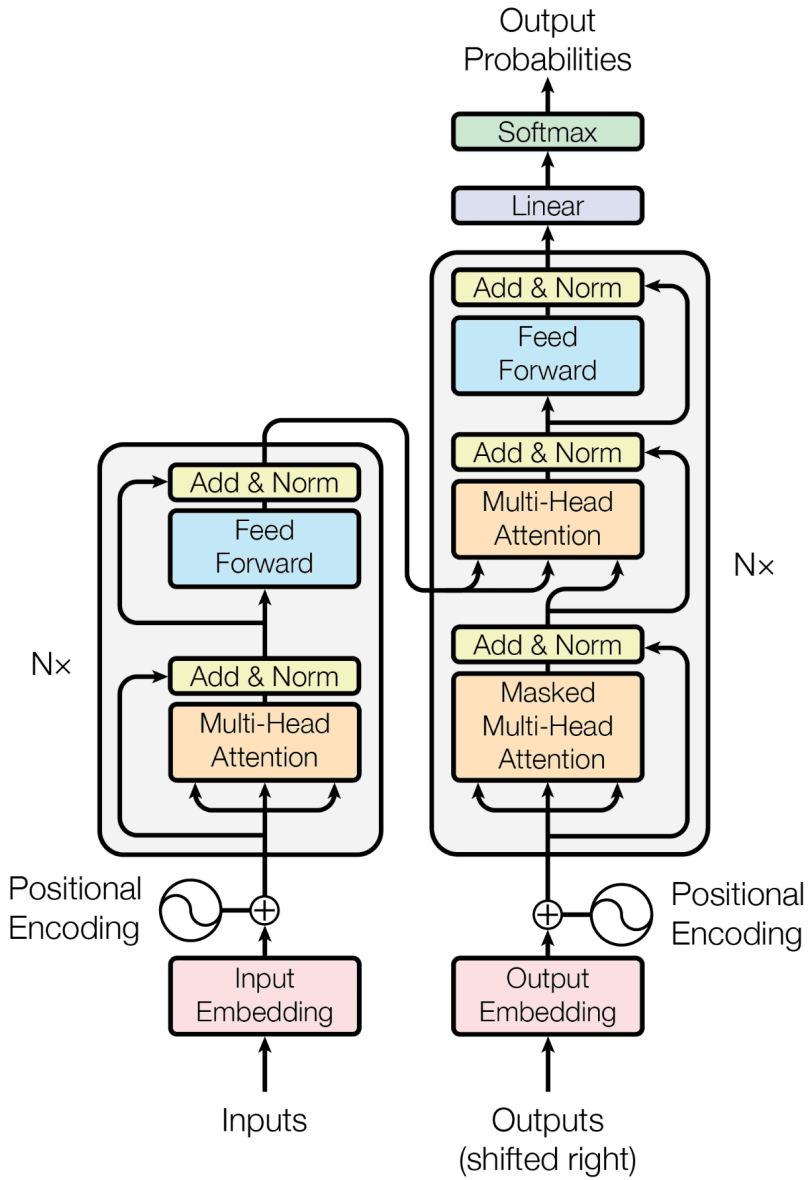


Transformers

Nacida en 2017, esta revolucionaria arquitectura de deep learning supuso una drástica mejora en el mundo del NLP gracias a sus mecanismos de atención, usando auto-atención o **self-attention,** eliminando la necesidad de arquitecturas recurrentes (RNNs) como las usadas hasta entonces para procesar lenguaje.

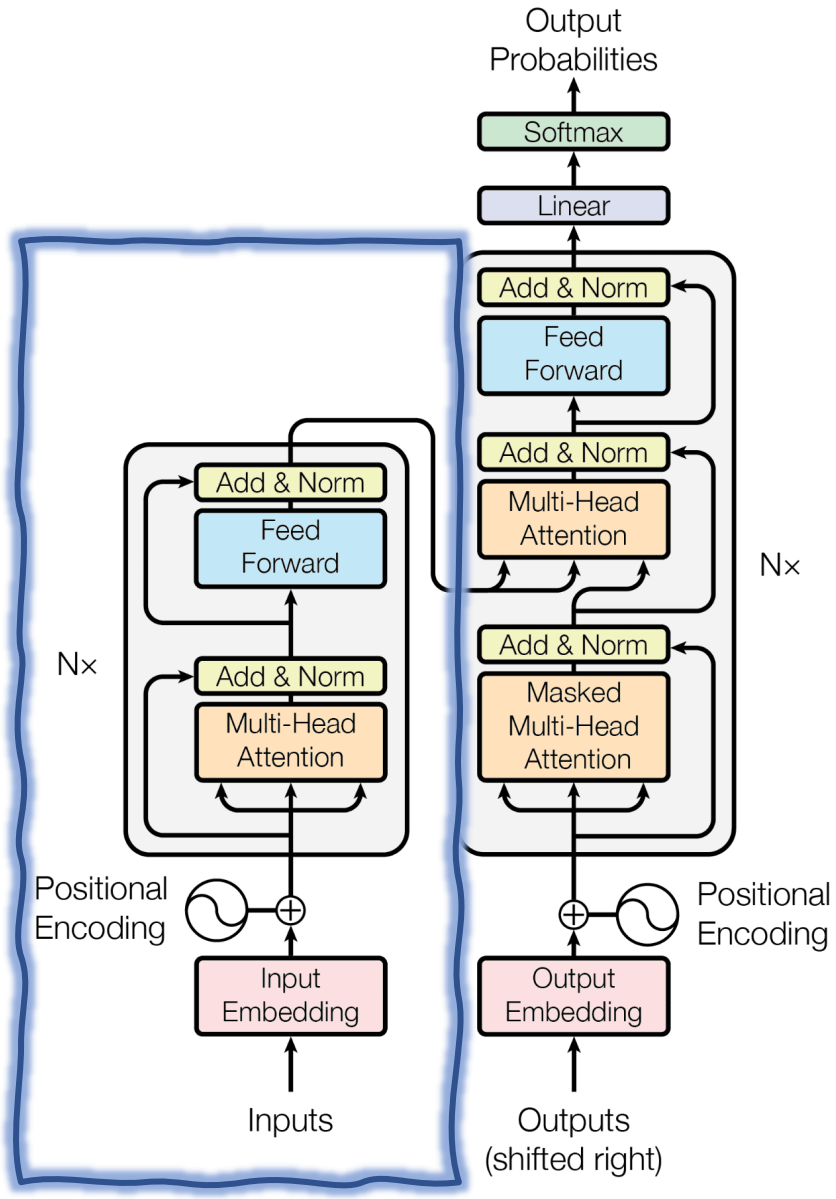
Consta de:

- Múltiples mecanismos de atención
- Conjuntos de capas (stacks) diferenciados: encoder y decoder; cada capa del encoder y decoder tiene:
    - Subcapas que alternan el mecanismo de atención con capas de neuronas secuenciales (feed forward).



Transformers

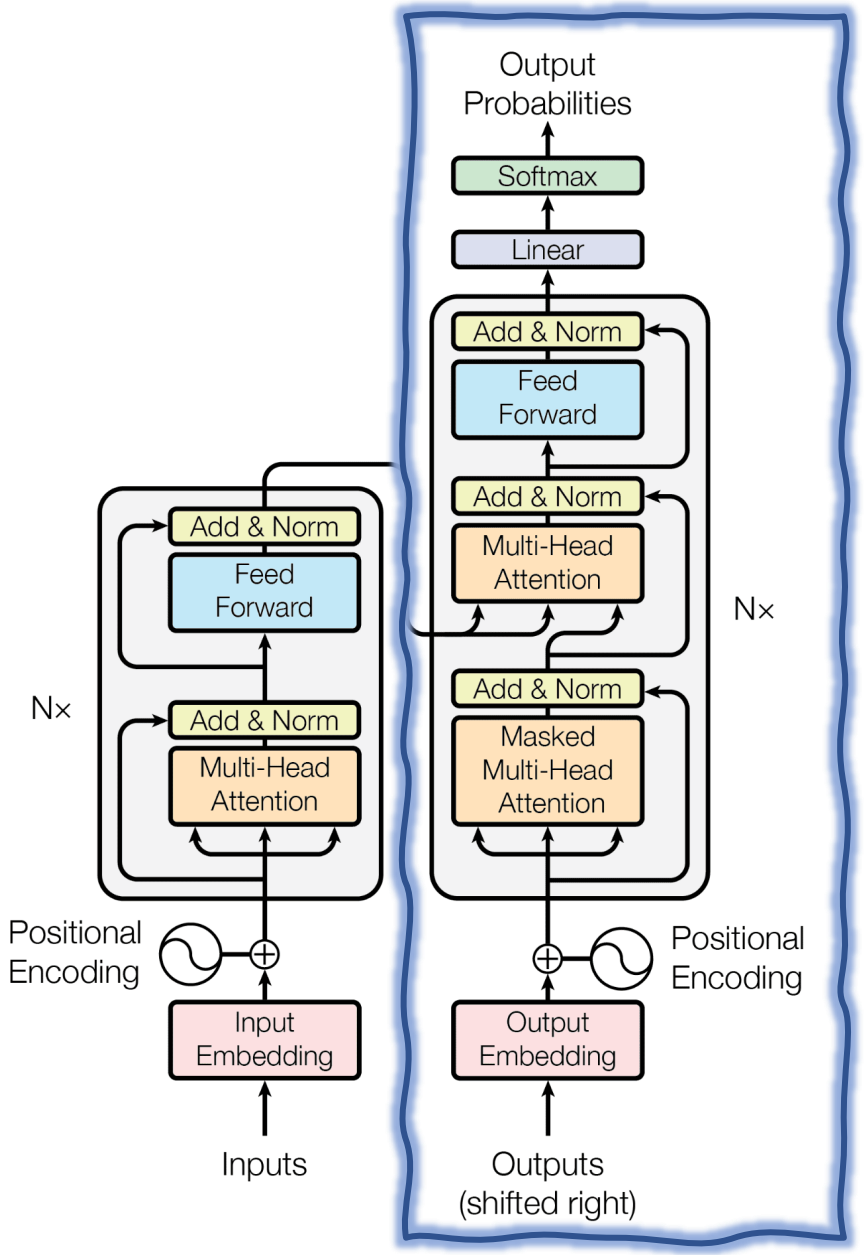
Encoder: gradualmente transforma la secuencia de entrada en una “representación oculta” (**hidden state representations)** que capturan el significado de la secuencia, relaciones y patrones lingüísticos de la misma con la mayor precisión posible.



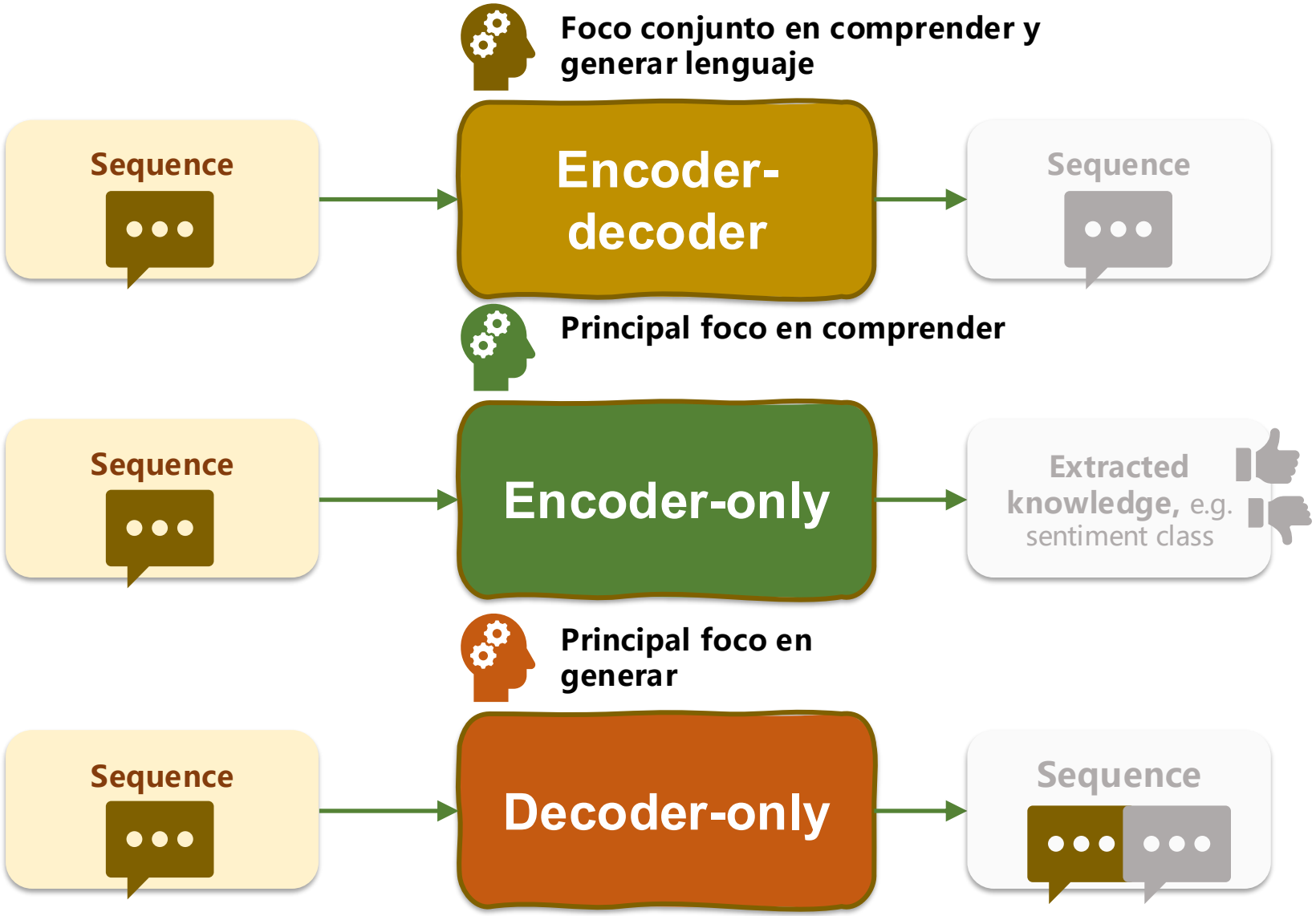
Transformers

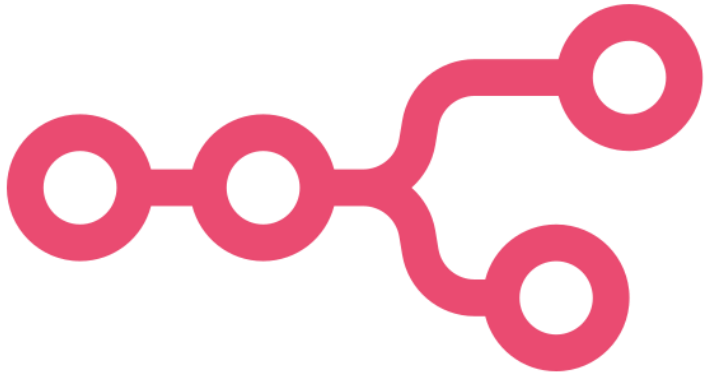
Decoder: toma las representaciones aprendidas por el encoder además de la salida generada hasta el momento para aprender a generar de forma acertada el siguiente token de la secuencia de respuesta.

Suele ir seguido de una capa final de neuronas con activación softmax → devuelve las probabilidades de la siguiente palabra/token a generar



Tipos de arquitectura de Transformer





n8n

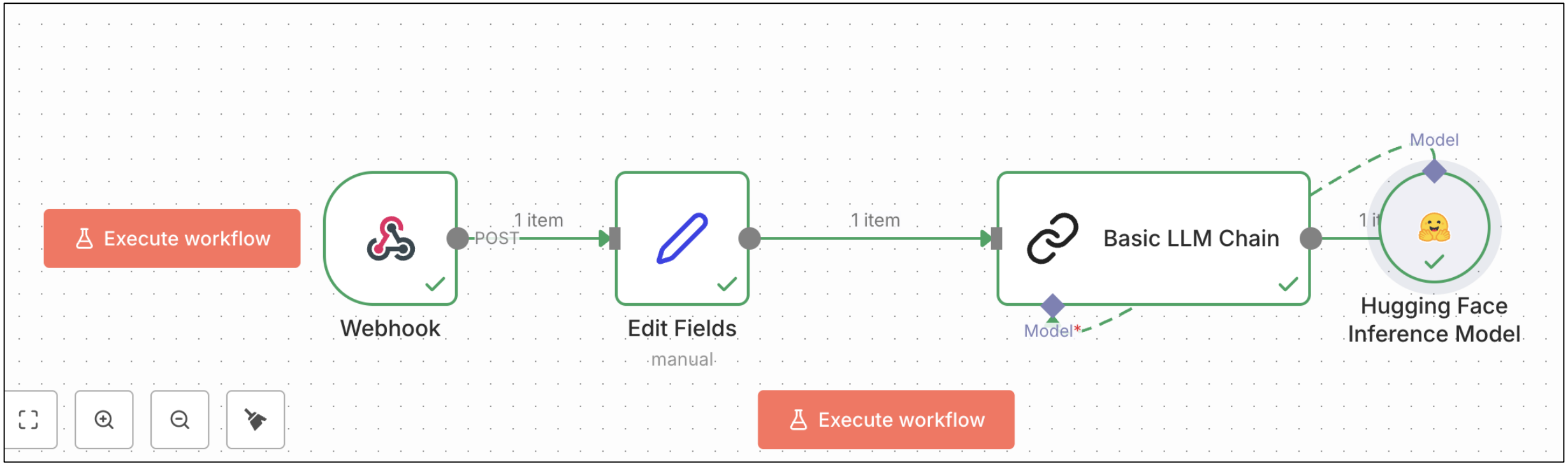




Hugging Face

Hugging Face es una plataforma que proporciona, entre otros servicios gratuitos, un gran repositorio de modelos de lenguaje pre-entrenados y listos para descargar y utilizar

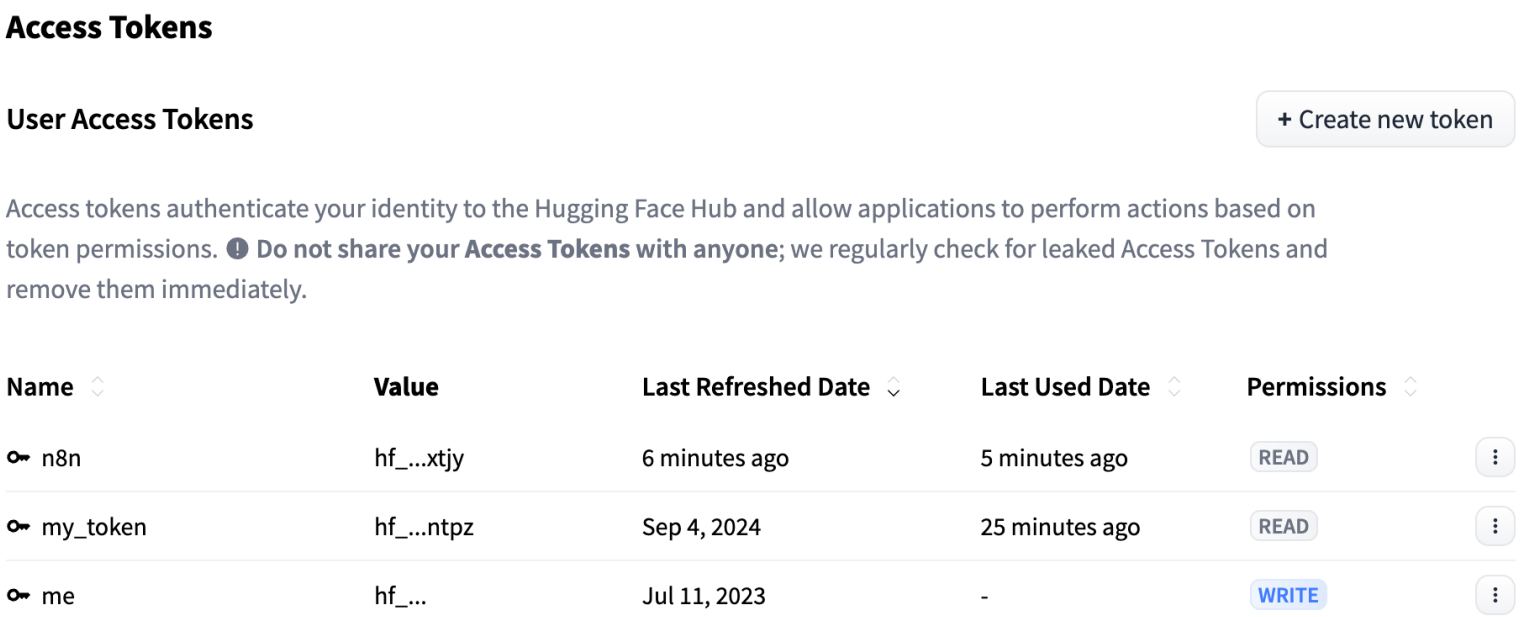
- El nodo a utilizar para llamar a un LLM de Hugging Face es: "**Hugging Face Inference Model**"
- Al crearlo, puedes eliminar el nodo "trigger" y dejar los otros dos, al igual que hicimos con LLMs de OpenAI.



LAS CLAVES QUE NECESITAS SABER: LA CREDENCIAL DE HUGGING FACE Y LA "API KEY"

- Para establecer la credencial de acceso a Hugging Face en n8n, debe registrarte en la web de Hugging Face (no importa si lo haces con una dirección de correo distinta a la que usaste para servicios de Google).
- Ve a tu icono de usuario arriba a la derecha → Access Tokens → Crea un nuevo token del tipo "Read" y cópialo.
- Guárdalo en un lugar seguro, y por supuesto pégalo en la ventana de credencial de n8n.

https://huggingface.co/settings/tokens

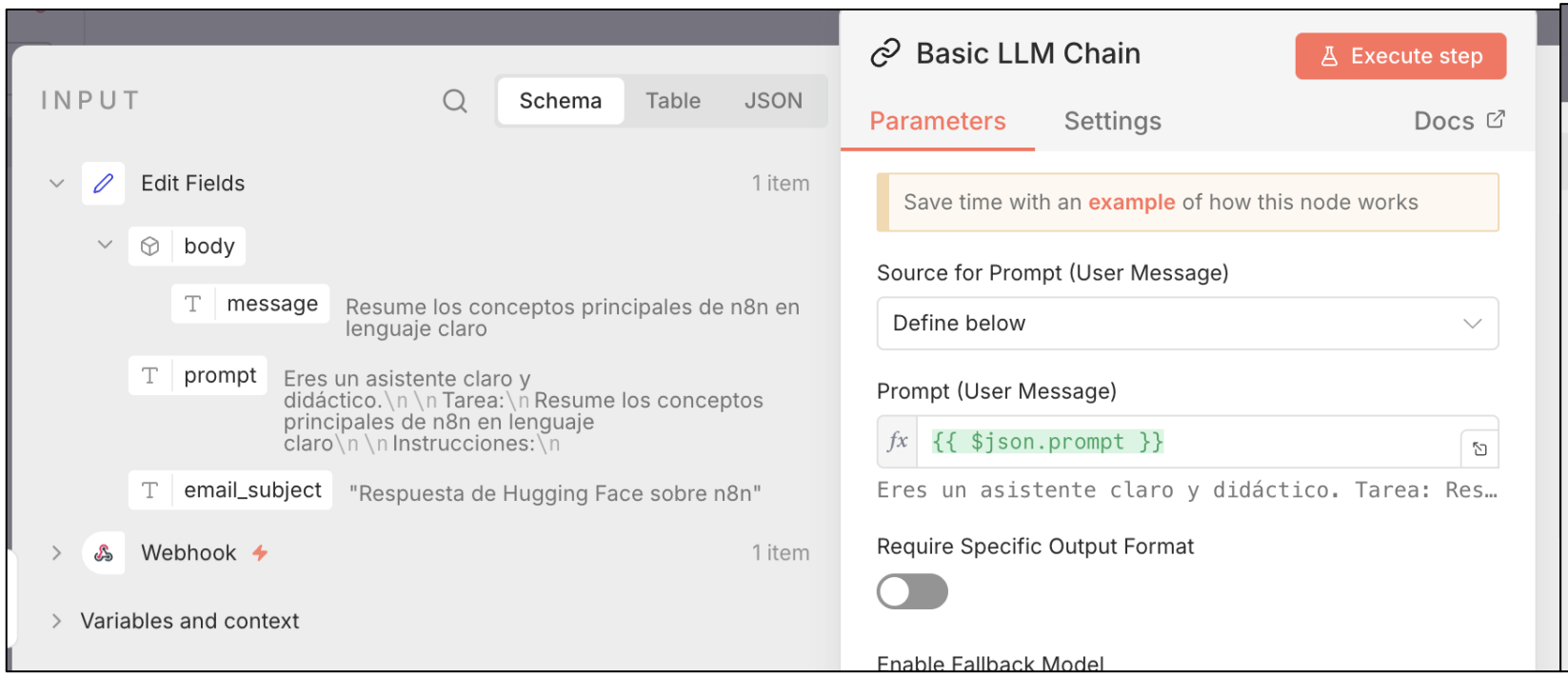


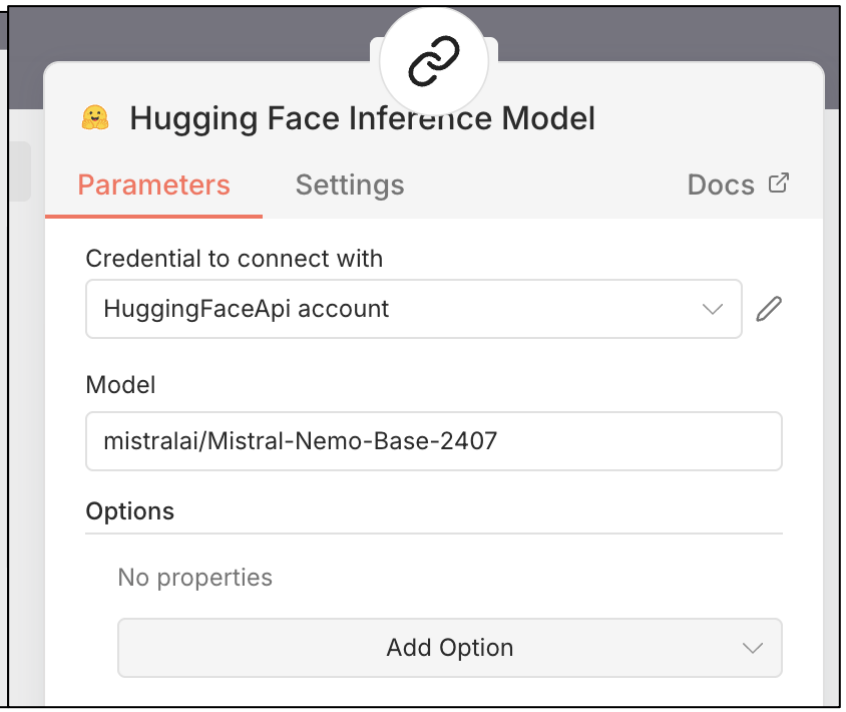
LAS CLAVES QUE NECESITAS SABER: CONFIGURAR NODO "BASIC LLM CHAIN"

- Asegúrate de conectarlo al nodo anterior de tu flujo, por ejemplo un nodo "Edit Fields (Set)"
- Elige como fuente del prompt "*define below*", y arrastra el campo "prompt" del objeto JSON de entrada

LAS CLAVES QUE NECESITAS SABER: CONFIGURAR NODO "HUGGING FACE INFERENCE MODEL"

- Vincúlalo con la credencial y especifica un nombre de modelo, igual que se haría desde Python





¿Por qué necesitamos RAG (Retrieval Augmented Generation)?

Los modelos de lenguaje grandes (LLMs) generan texto a partir de los datos sobre los que fueran entrenados. Por muchos y muy actuales que fueran estos datos en el momento de entrenarlos, pasado un tiempo, ya no tienen acceso directo a información totalmente actualizada, y esto es crítico en un sistema como son los LLMs.

Problemas:
Información obsoleta.
Alucinaciones: el modelo puede inventarse respuestas que parezcan plausibles pero no veraces, lo cual es peligroso en casos de uso críticos.
Alto costo de re-entrenar el modelo con frecuencia para mantener actualizada su “base de conocimiento”.

¿SOLUCIÓN? ¡QUE EXISTA UNA BASE DE CONOCIMIENTO EXTERNA AL MODELO!

¿Qué nos aporta una arquitectura RAG?

Retrieval Augmented Generation (RAG) combina dos grandes elementos

1.Recuperación de información external (Retrieval) → como los motores de búsqueda tradicionales o las Vector DBs: bases de datos vectoriales con representaciones de textos (embeddings).

2.Generación de texto/respuestas con LLMs que usan la información recuperada como contexto que enriquecerá el *prompt* del usuario.

🎯 **Beneficios principales del RAG**

✔ **Respuestas más precisas y fundamentadas**
✔ **Conocimiento actualizado sin reentrenar el modelo**
✔ **Menos alucinaciones**, porque se basa en datos reales
= Mejor **confianza y utilidad en aplicaciones reales**

¿Cómo es la arquitectura básica de un sistema RAG?

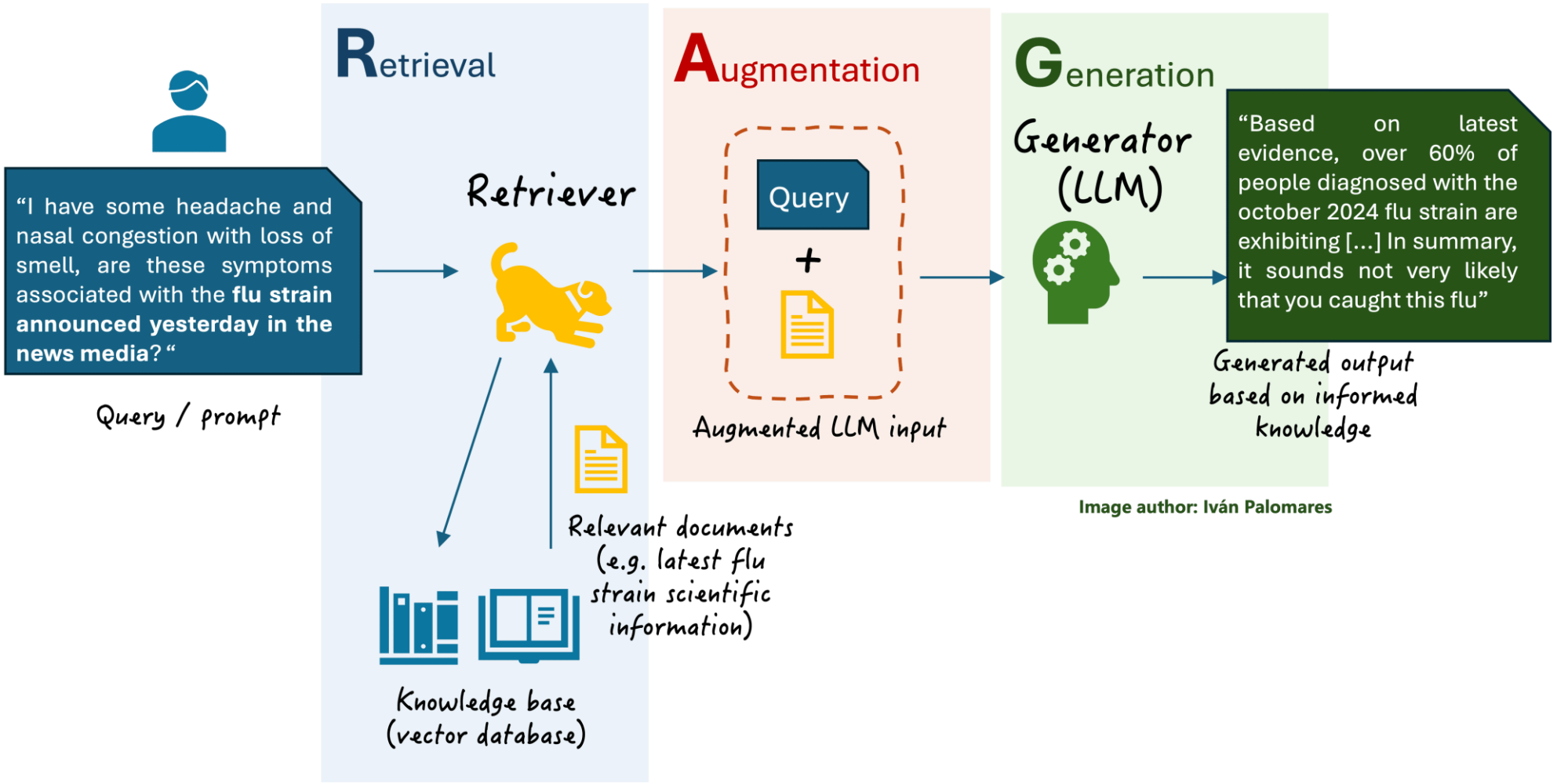


Image author: Iván Palomares

¿Cómo es la arquitectura básica de un sistema RAG?

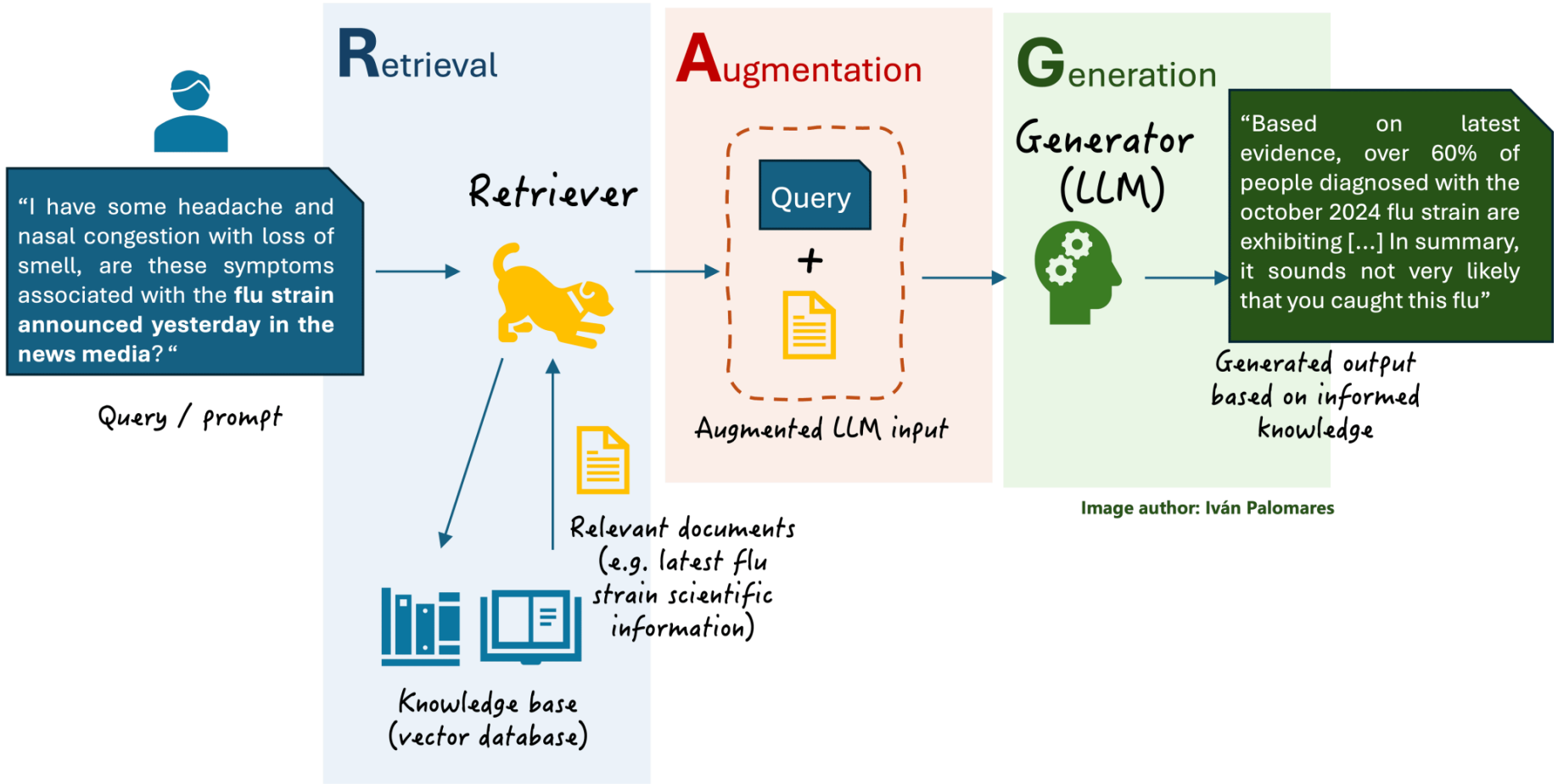
LLM (Generator)
Modelo de lenguaje que genera respuestas, recibiendo un texto de entrada.

Vector database o knowledge base
Almacena *embeddings* de documentos o textos.

Recuperador / Retriever
Se encarga de encontrar y extraer los textos más relevantes de la base de conocimiento, dada una consulta (*query*)
Usa una búsqueda basada en similitud entre vectores: *coseno, nearest neighbor, etc.*

Flujo Típico (3 Etapas)

1. **Retrieval (Recuperación): l**a consulta del usuario se transforma en vector y se usa para buscar textos relevantes en la vector DB.
2. **Augmentation (Aumento) del prompt:** Los fragmentos relevantes recuperados se **anexan/contextualizan** con la consulta original.
3. **Generation (Generación):** El LLM recibe la consulta aumentada y genera el texto final, aprovechando contexto actualizado



¿Qué hace tan potente a esta arquitectura RAG?

Recuperación semántica: la búsqueda que realiza el *retriever* se basa en significado -a nivel de embeddings de textos- no en palabras exactas.

Conocimiento externo desacoplado del LLM: buena parte de la información reside fuera del LLM. Actualizar la base de conocimiento es mucho más sencillo que reentrenar el modelo.

Menos alucinaciones: las respuestas del LLM se fundamentan en documentos reales en gran medida.

RAG Clásico = LLM + búsqueda inteligente + contexto relevante

¿Qué es Agentic AI (IA de agentes)?

Es un paradigma bajo el cual un sistema de AI actúa como agente con un cierto grado de autonomía para realizar acciones por sí mismo, dejando de ser puramente “reactivo” como por ejemplo un LLM u otro modelo de ML.

Un agente de IA:

- Tiene un objetivo
- Puede tomar decisiones de qué acción o acciones realizar a continuación
- Utiliza herramientas: APIs para “hablar” con otras apps, bases de datos, flujos, etc.
- Observa resultados y adapta su comportamiento de forma acorde.

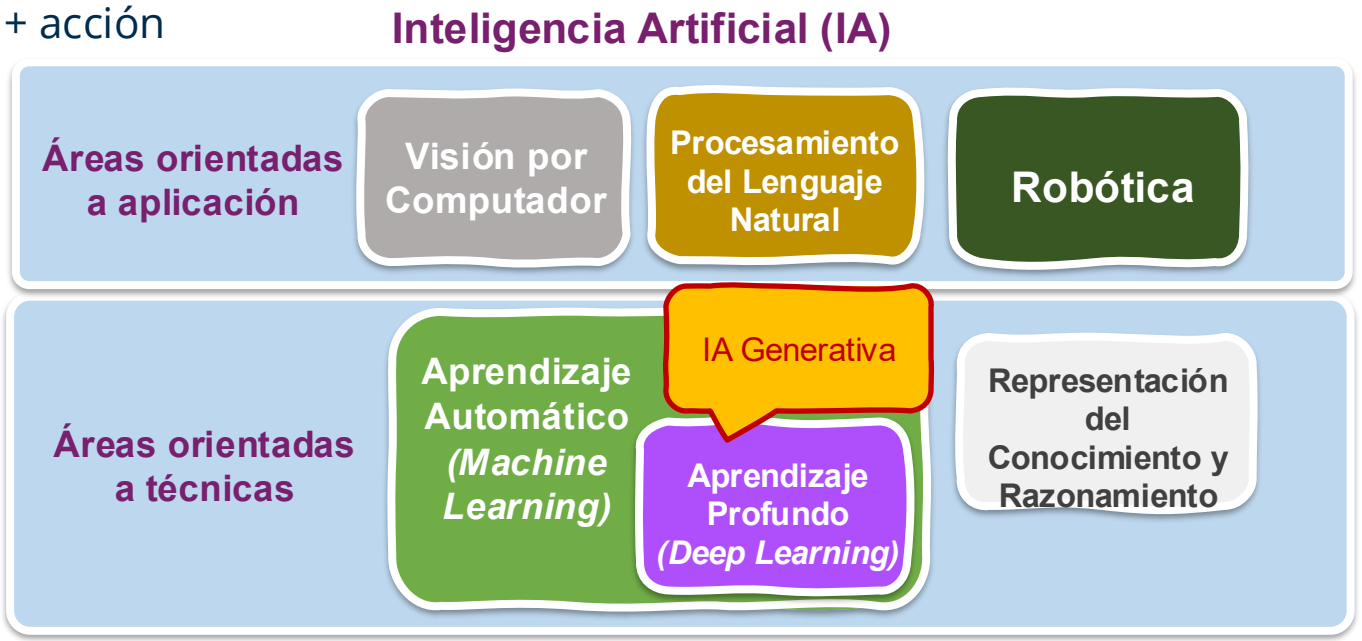


De Modelos de IA/ML "reactivos" a agentes "proactivos"

- **Modelos de ML y DL más clásicos:** predicen, estiman, clasifican, etc., a partir de ejemplos de datos de entrada.
- **LLMs**: capaces de entender y generar lenguaje humano.
- **RAG**: combinan LLMs + conocimiento externo
- **Agentes**: LLMs (o RAG) + razonamiento + acción

Con sus capacidades propias de razonamiento y acción, **los agentes** pueden orquestar la interacción entre modelos y herramientas de forma más inteligente y flexible

→ **Eligen qué hacer en función del contexto**



COMPONENTES BÁSICOS DE UN AGENTE

- **Objetivo**: qué quiere logar.
- **Memoria / contexto**: qué conocimiento tiene hasta el momento.
- **Razonamiento**: decidir sobre la siguiente acción a realizar
- **Herramientas:** acciones disponibles
- **Ciclo lógico:** pensar (razonar) → actuar → observar resultados → pensar (...)

VALOR DE LOS AGENTES

- Automatización y orquestación inteligente de tareas complejas y abiertas.
- Se reduce la necesidad de flujos 100% predefinidos.
- Mejor escalabilidad ante variabilidad, decisiones intermedias, e incertidumbre.

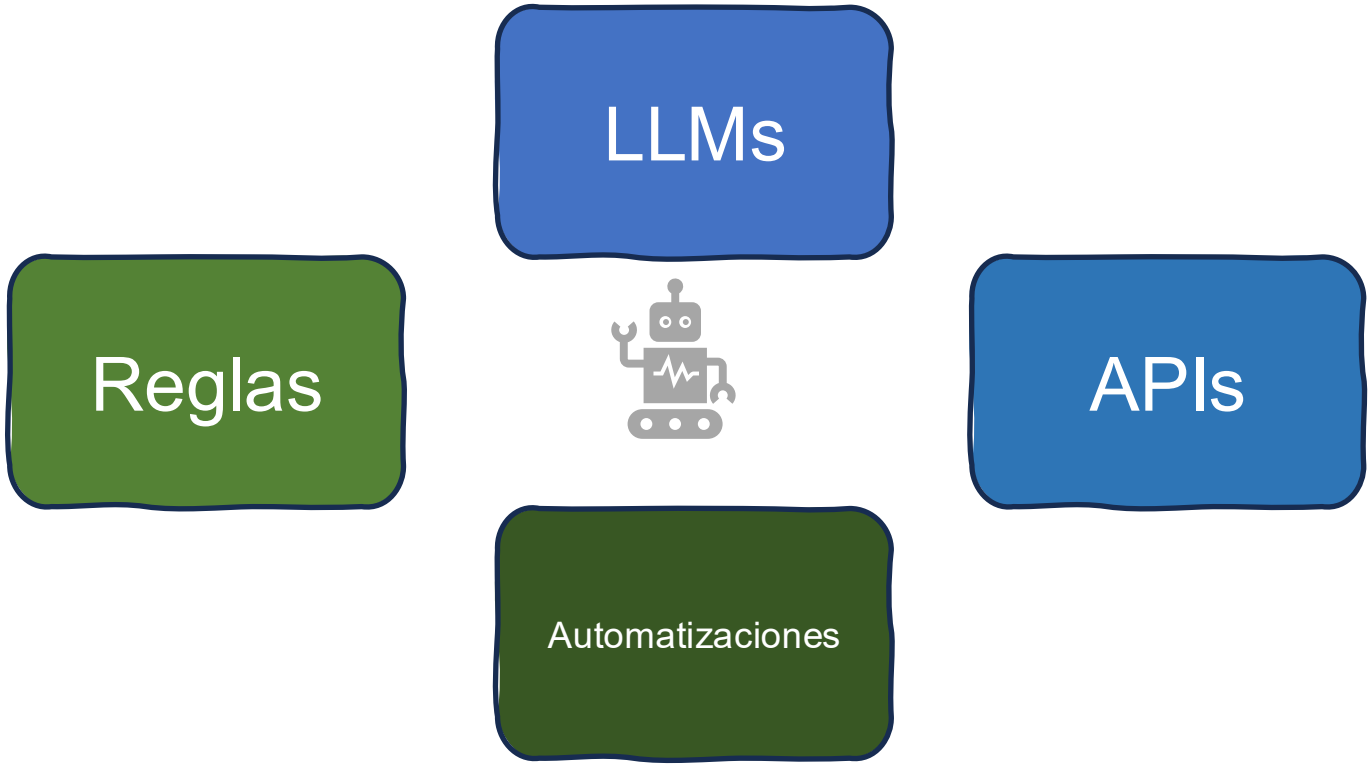
RELACIÓN DE LOS AGENTES CON SISTEMAS DE IA

Los agentes no suponen un reemplazo de los LLMs y otros modelos de IA.

Su función es potenciar los modelos existentes y el contexto o caso de uso que los rodea.

- LLMs /AI → cerebro
- Agente → 2º cerebro + "manos"

Pueden integrar

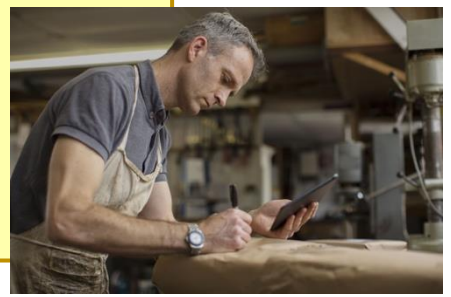


EJEMPLOS TÍPICOS DE AGENTIC AI

EJEMPLO 1: Asistente que...
- Analiza un e-mail entrante.
- Decide si responder, archivar, o escalar brindándoselo a un interventor humano.

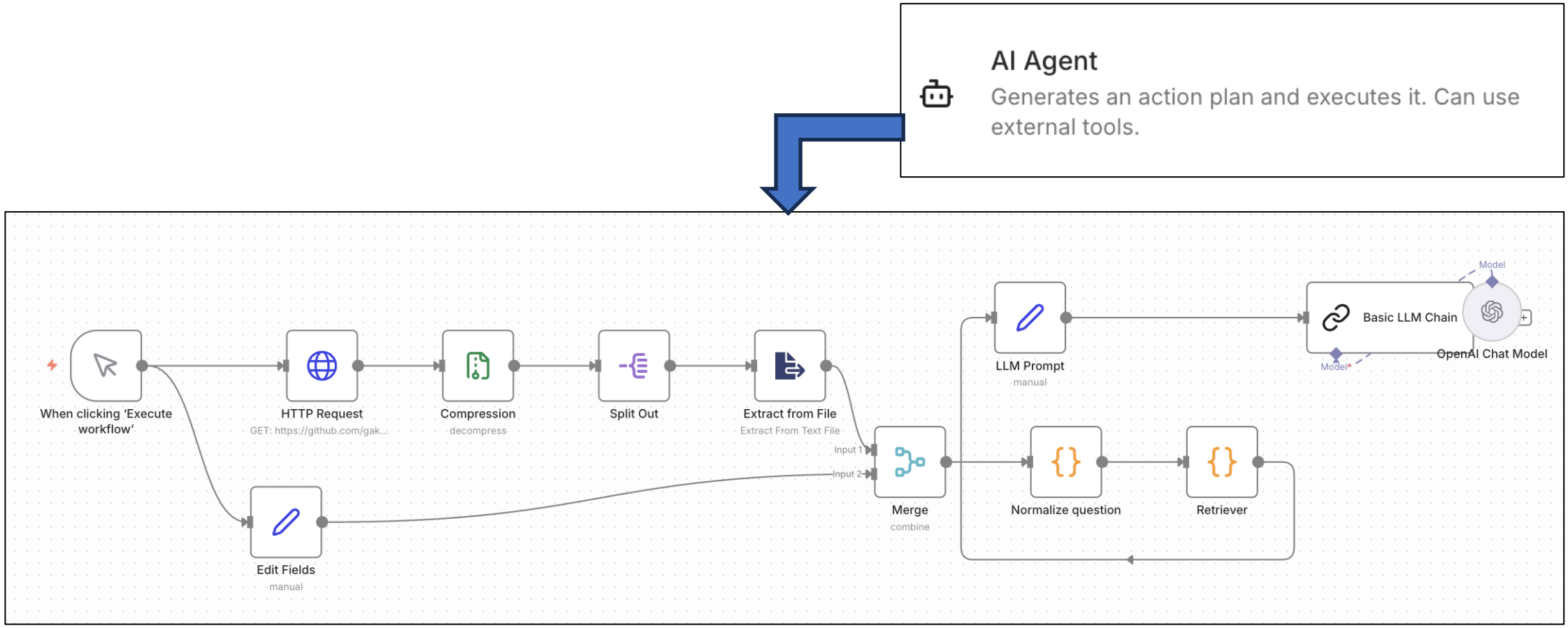


EJEMPLO 2: Sistema que ...
- Recibe un pedido de un cliente.
- Consulta datos de inventario.
- Genera automáticamente una propuesta personalizada en base al inventario disponible.
- Envía confirmación y crea un registro de la actividad realizada en un CRM



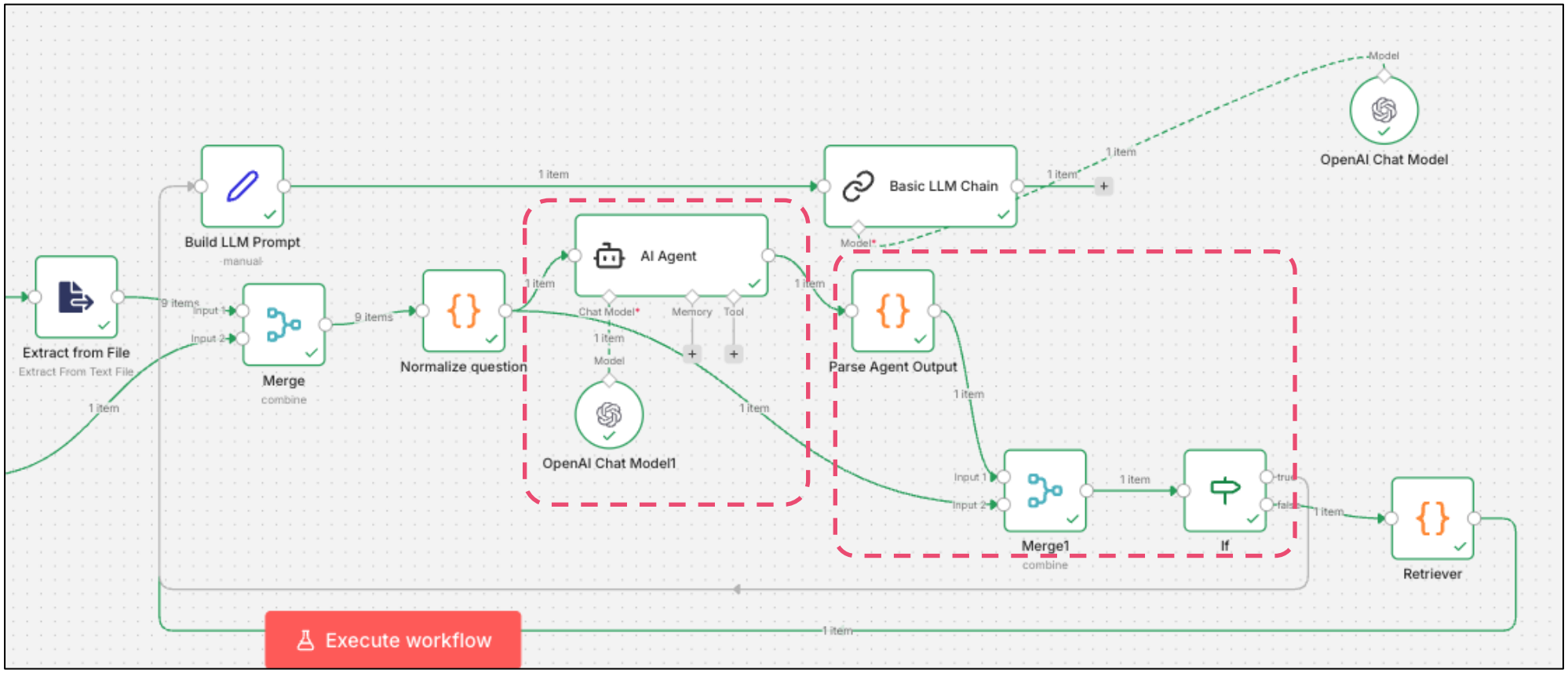
¿QUÉ APORTA EL INTRODUCIR UN AGENTE IA EN UN FLUJO INICIALMENTE DETERMINISTA?

- Sin el agente, nuestro flujo para un sistema RAG (ver imagen inferior) se ejecuta siempre igual, con decisiones manualmente codificadas en el propio código y lógica de ciertos nodos.



¿QUÉ APORTA EL INTRODUCIR UN AGENTE IA EN UN FLUJO INICIALMENTE DETERMINISTA?

- Con un agente (conectado a su propio modelo LLM), el sistema puede razonar antes de ejecutar.
- En un flujo como este, el resultado del razonamiento del agente se "proyecta" en un posterior nodo condicional (IF), donde se decide si se usará el retriever para filtrar documentos relevantes o no.



¿CÓMO LA LÓGICA DEL AGENTE SE PROYECTA EN EL FLUJO DE N8N?

- El agente, mediante su propio LLM, analiza la intención de la pregunta y construye una decisión o salida estructurada en JSON.
- No ejecuta acciones, pero dice a n8n cómo actuar después

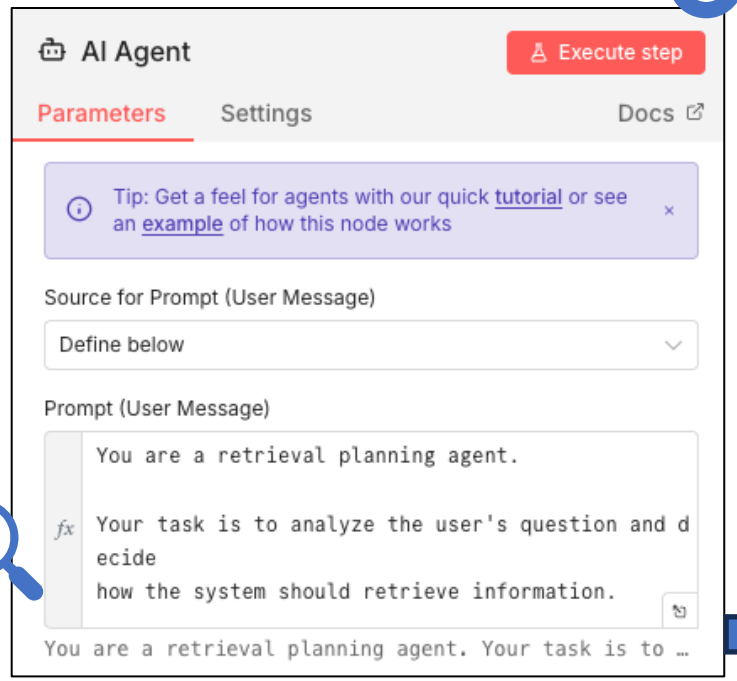
```
You are a retrieval planning agent.

Your task is to analyze the user's question and decide
how the system should retrieve information.

You must return ONLY a valid JSON object with the following
fields:
- retrieval_strategy: "keyword" or "none"
- top_k: number
- reason: short explanation

Rules:
- If the question can be answered from general knowledge,
  use retrieval_strategy = "none".
- If the question refers to specific documents or regions,
  use retrieval_strategy = "keyword".
- Keep the JSON concise.
- Do not include any extra text outside the JSON.

User question:
{{ $json.question }}
```

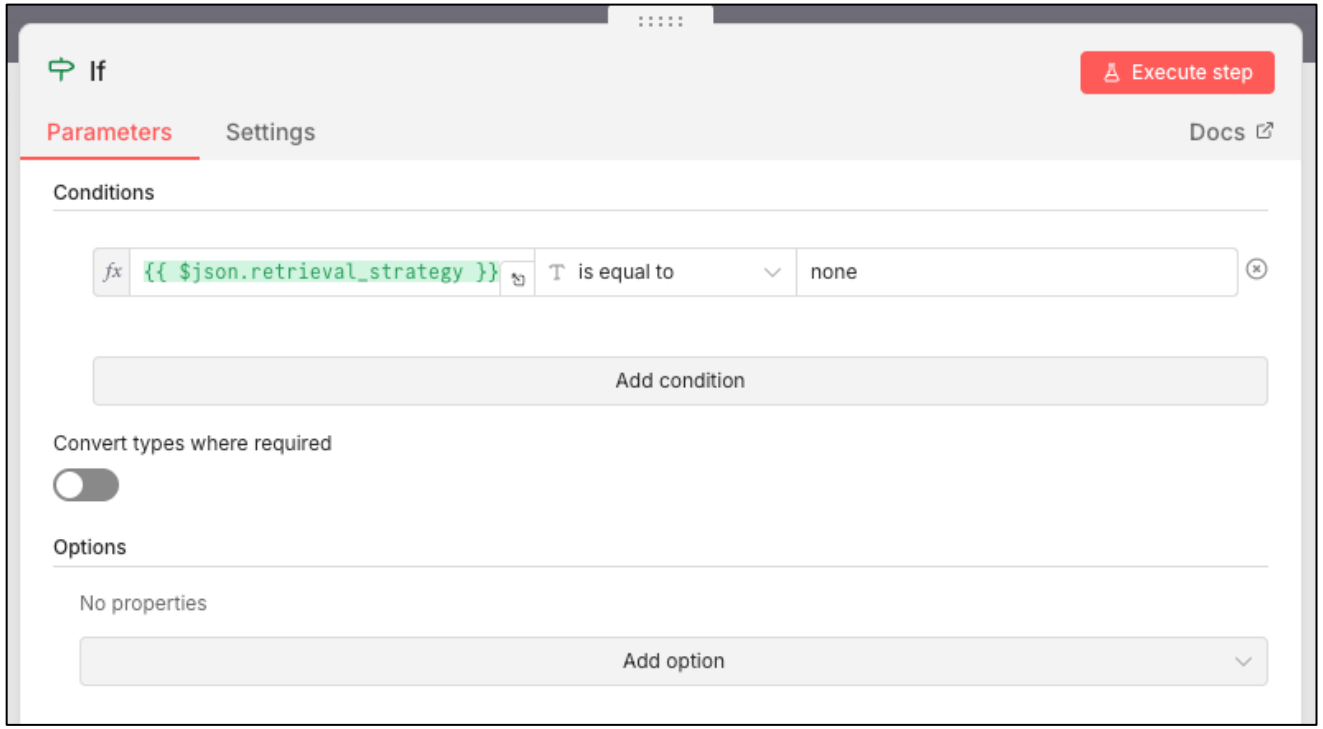


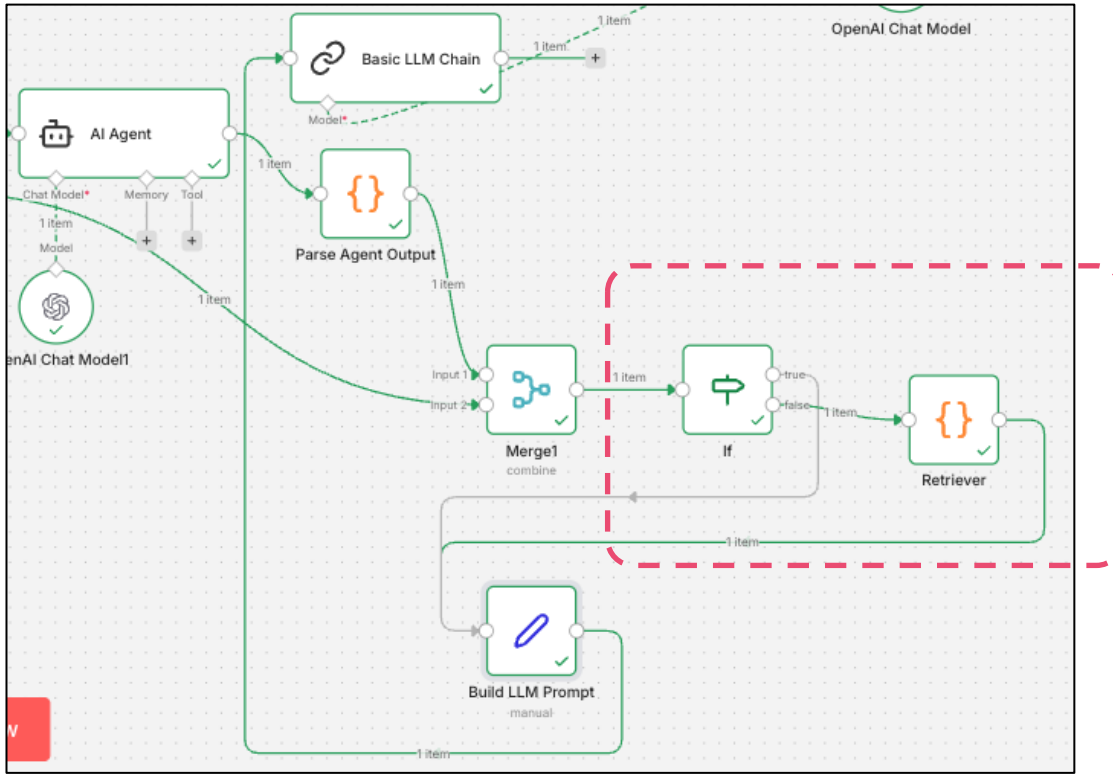
Salida del agente

[
{
"output": "```json\n{\n \"retrieval_strategy\": \"keyword\",\n \"top_k\": 5,\n \"reason\": \"The question refers to specific information from provided documents about Southeast Asian countries with islands and beaches.\"\n}\n```"
}
]

¿CÓMO LA LÓGICA DEL AGENTE SE PROYECTA EN EL FLUJO DE N8N?

- La información emitida por el agente, por ejemplo, la estrategia inteligente a emplear (con o sin filtrado de documentos por parte del *retriever*), se "proyectará" en un nodo posterior.
- Por ejemplo, en un simple nodo condicional, donde elegiremos un camino u otro (pasar o no por el *retriever*), según un campo especificado de los provenientes del agente.





RESUMEN DE LOS SUBNODOS DE UN AGENTE N8N

Chat Model (LLM)

Es un LLM conectado al nodo agente quien realmente razona y toma decisiones.

- Interpreta la pregunta
- Decide si usar memoria
- Decide si llamar a una tool
- Redacta la respuesta final

Ejemplos:
OpenAI Chat
Azure OpenAI Chat
Otros chat LLMs compatibles

OBLIGATORIO

Memory

Almacenamiento de contexto conversacional. Conceptualmente relacionado con la memoria a corto y largo plazo que veremos después

- Mantener historial de mensajes.
- Permitir coherencia entre ejecuciones.
- Simular capacidades de “memoria” del agente

Ejemplos:
- Buffer memory
- Window memory

OPCIONAL

Tool

Funciones externas que el agente puede ejecutar

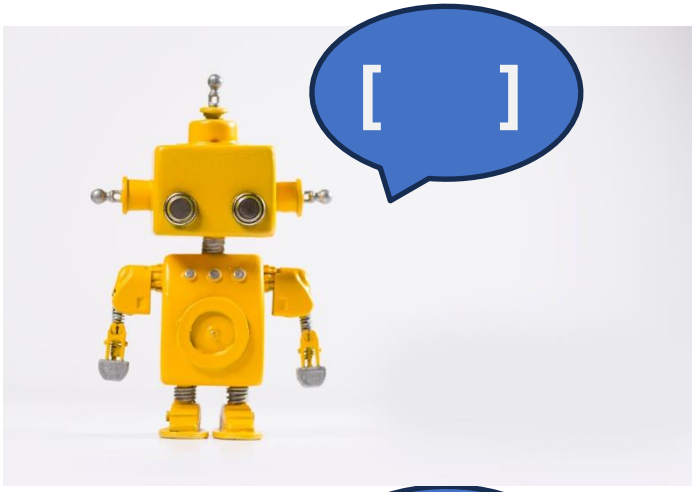
- Acceso a información externa.
- Ejecución de acciones en BD, HTTP, cálculos, APIs, etx.

Ejemplos:
- Wikipedia
- HTTP
- BBDDs

OPCIONAL

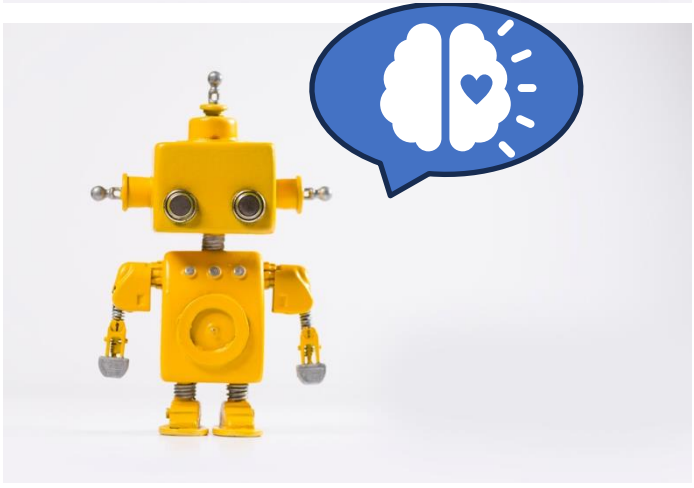
¿Qué es la memoria en LLMs y agentes?

- Un mecanismo para **conservar y reutilizar contexto de llamadas anteriores**, ni ciñéndonos únicamente a la actual llamada al modelo.
- Permite al sistema "recordar" interacciones, información y estados previos.



En un modelo sin memoria…

- Cada prompt es independiente.
- No se recordará nada de conversaciones pasadas ni decisiones del agente previas.



En un modelo con memoria…

- Las respuestas podrán ser más coherentes y personalizadas.
- El modelo/agente tiene la capacidad de mantener una **continuidad** en los flujos conversacionales y procesos complejos.

Memoria a corto y largo plazo en n8n

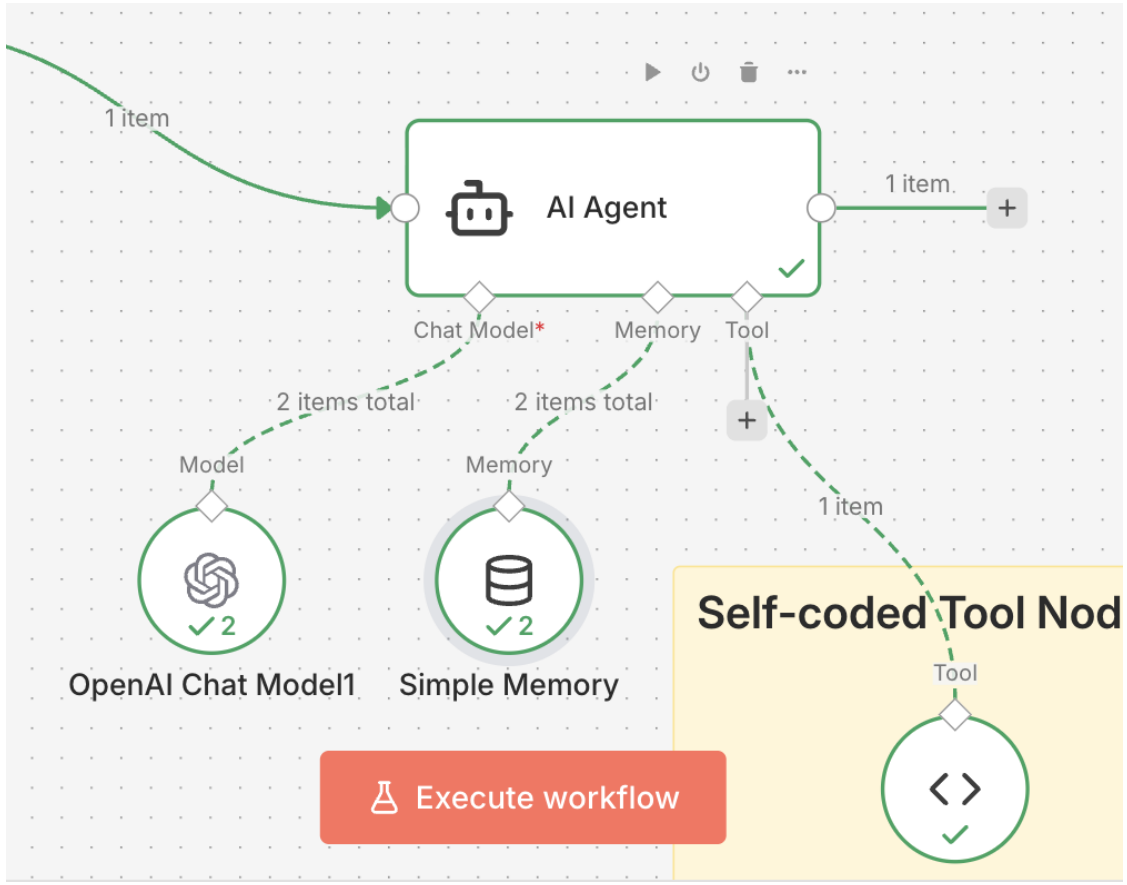
- Se añade como subnodo dentro de los nodos de tipo **AI Agent**.
- Lo más sencillo es mediante un subnodo "**Simple Memory**"

El nodo "Simple Memory"

- Dota al agente de un mecanismo de memoria básica y automática.
- Guarda el historial *reciente* de la conversación con el LLM del agente y lo reinyecta en prompts sucesivos.
- No incorpora razonamientos avanzados, solo retiene contexto a corto plazo.

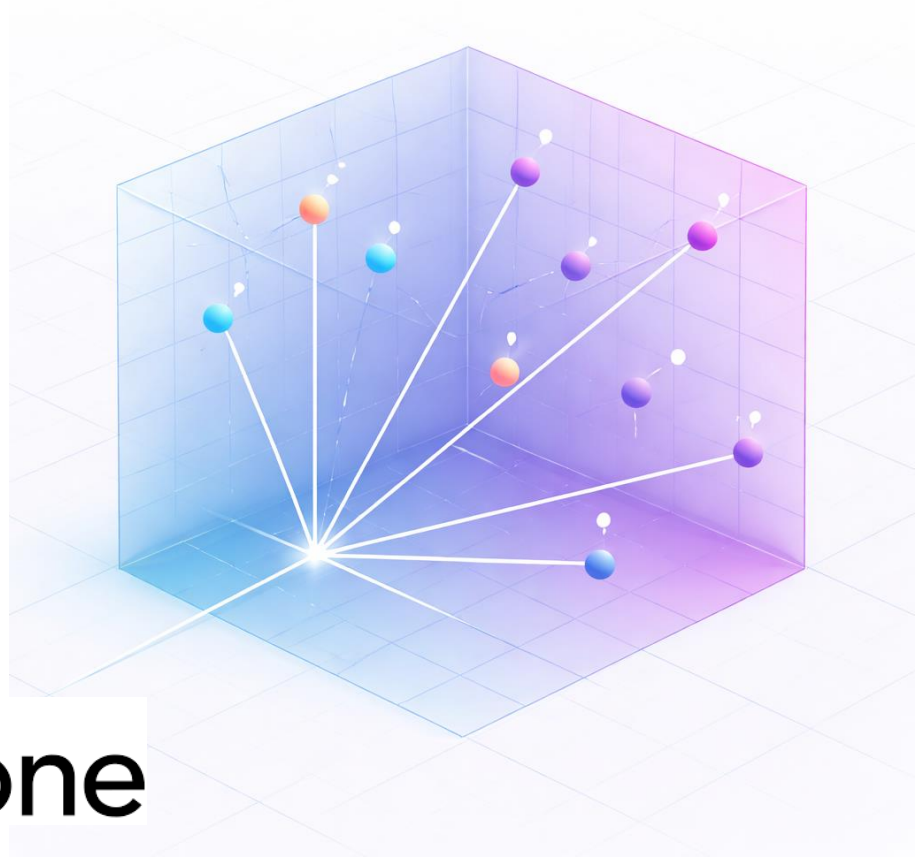
¿Y para memorias a largo plazo?

- Se requerirá enfoques más avanzados, usando BBDDs externas como SQL/NoSQL, o bien BBDD vectoriales gestionadas en el flujo, pero no a través de Simple Memory



BBDDs vectoriales en sistemas RAG

- Son BBDD diseñadas específicamente para almacenar, recuperar, y realizar búsquedas sobre vectores numéricos (embeddings), normalmente asociados a representaciones de documentos/textos en lenguaje natural.
- Cada *embedding* encapsula la semántica de un texto
- También existen *embeddings* de imagen, audio, etc.
- En cualquier caso, hacen posible la **búsqueda por similitud a nivel semático**, no por coincidencia de *keywords*.



Valor de los Embeddings en RAG

- Permiten encontrar información conceptualmente similar, incluso sin que se usen exactamente las mismas palabras.
- Mucho mejores resultados ante consultas de usuario ambiguas, en lenguaje natural, y con muchos sinónimos y variaciones semánticas

FAISS (FACEBOOK AI SIMILARITY SEARCH)

Weaviate

Pinecone

CLAVES PARA LA INTEGRACIÓN DE BBDD VECTORIALES EN N8N

N8n es, como sabemos, el orquestador de un flujo, incluyendo flujos RAG

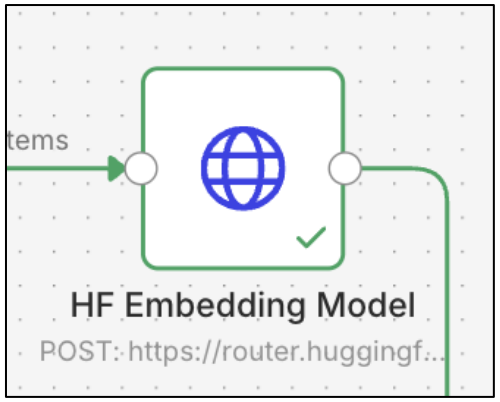
- Podemos conectarlo a modelos de creación de embeddings
- Ejemplo: los *sentence-transformer*s de Hugging Face y otros similares.
- También se conecta a BBDDs vectoriales.

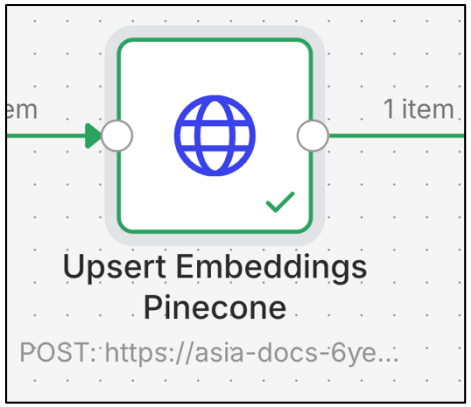
Pero …. lo hacemos desde nodos estándar (a menudo HTTP) → no con nodos dedicados de app concreta

Desde n8n podremos…

- Generar embeddings usando modelos abiertos: Hugging Face, APIs de OpenAI, etc.
- Guardar embeddings con metadatos asociados en una BBDD vectorial como por ejemplo Pinecone.
- Consultar la BBDD vectorial y recuperar textos relevantes dada una consulta.
- Construir un contexto y enviarlo a un LLM para que genera una respuesta “RAG” al usuario.

Una combinación de nodos HTTP tipo GET/POST, de código, y otros básicos para pre-procesar y post-procesar datos en el flujo, será normalmente la fórmula del éxito



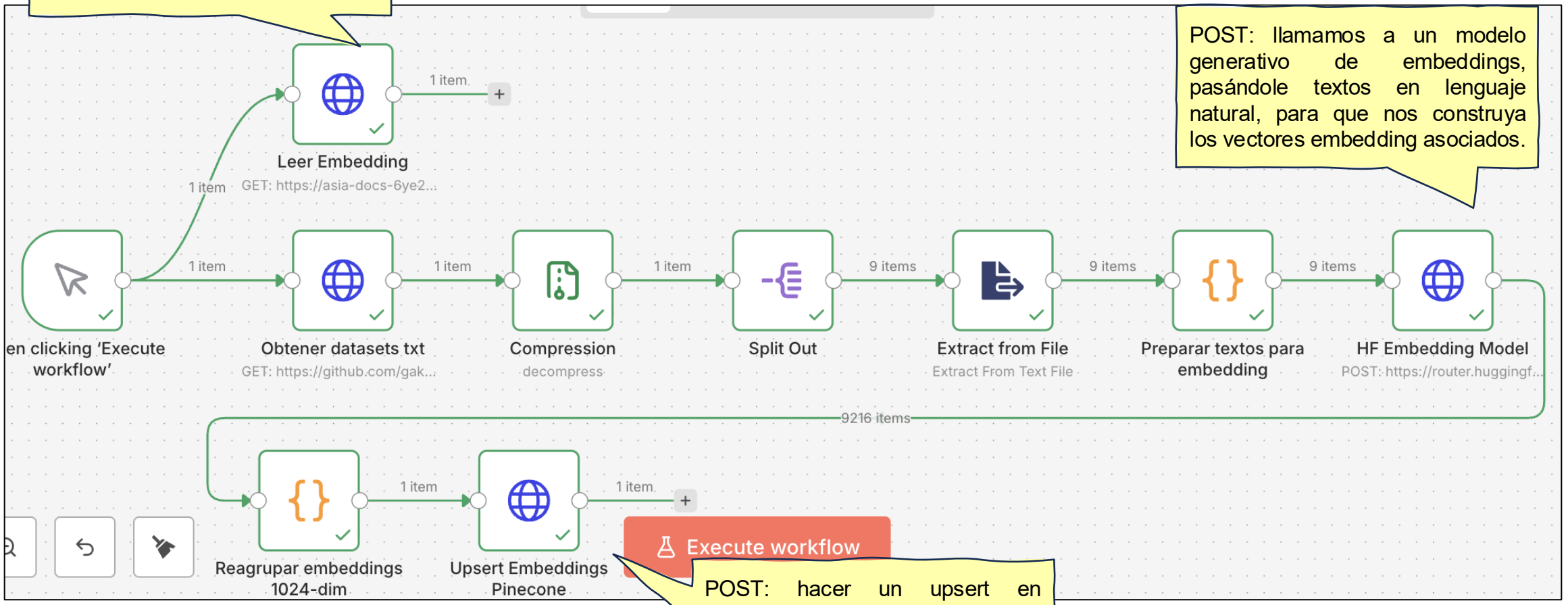


GET: Lectura de embedding(s) de una BBDD o Index ya existente en Pinecone

Pinecone

Hugging Face

POST: llamamos a un modelo generativo de embeddings, pasándole textos en lenguaje natural, para que nos construya los vectores embedding asociados.



POST: hacer un upsert en Pinecone, que no es otra cosa que subir nuestros recién generados embeddings para almacenarlos en un Index (BBDD vectorial)

Pinecone